

Tijdscomplexiteit

1001WETALC

Deterministisch polynomiale tijd (PTIME)

Satisfiability problemen

Niet-deterministisch polynomiale tijd (NP)

Reductie, compleetheid en hardheid

De stelling van Cook-Levin

NP-complete problemen

Efficiënt = polynomiale tijd

Houdt dit steek?

- ▶ Op deze manier kunnen we bijvoorbeeld n^3 en 2^n onderscheiden.
- ▶ Exponentieel $\approx$ exhaustieve enumeratie.
- ▶ Polynomiaal = iets slimmer.
 - ▶ Polynomiaal $\approx$ oplosbaar in praktijk.
 - ▶ Alle deterministische berekeningsmodellen zijn *polynomiaal equivalent* (d.w.z. dat deze modellen kunnen in elkaar worden getransformeerd in polynomiale tijd)
 - ▶ Het geeft een *fundamenteel inzicht* in een probleem als we het kunnen oplossen in polynomiale tijd.
 - ▶ Meeste algoritmen in polynomiale tijd hebben kleine exponent in n (bijv. n , $n^{2/3}$, n^3 , ...).

Definition

Stel, $t : \mathbb{N} \rightarrow \mathbb{N}$ is een functie. We definiëren de **deterministische tijdscomplexiteitsklasse** $\text{DTIME}(t(n))$ als:

$$\text{DTIME}(t(n)) = \{L \mid \text{de taal } L \text{ wordt beslist door een } \mathcal{O}(t(n))\text{-tijd} \\ \text{deterministische Turing machine met één tape}\}.$$

Voorbeeld

De taal van palindromen zit in $\text{DTIME}(n^2)$.

Veronderstel dat je computer **100 miljoen stappen** doet per seconde.

De onderstaande tabel toont de **CPU tijd** voor verschillende functies $t : \mathbb{N} \rightarrow \mathbb{N}$.

n	$t(n) = n$	$t(n) = n^2$	$t(n) = n^3$	$t(n) = 2^n$
10	0.01 microsec	0.1 microsec	1 microsec	1 microsec
20	0.02 microsec	0.4 microsec	8 microsec	1 millisecc
50	0.05 microsec	2.5 microsec	125 microsec	13 dagen
100	0.1 microsec	10 microsec	1 millisecc	4×10^{13} jaren

Polynomiaal versus exponentieel maakt echt wel een verschil.

PTIME = de klasse van alle **talen** die beslist kunnen worden door een deterministische TM met één tape, en dit in **polynomiale tijd**.

Definition

$$\text{PTIME} = \bigcup_k \text{DTIME}(n^k).$$

Let op: We beschouwen steeds **beslissingsproblemen**!

Wanneer je wil aantonen dat een probleem in PTIME (of een ander complexiteitsklasse) zit zullen we **niet steeds een TM in detail beschrijven**.

In plaats daarvan beschrijven we de stappen van de TM (of algoritme) op een iets meer **“natuurlijke”** manier en tonen dan aan dat alle stappen tezamen $\mathcal{O}(n^k)$ zijn voor een zeker $k \geq 0$ (of een ander functie afhankelijk van de complexiteitsklasse).

Je moet wel steeds in staat zijn om de **TM in alle details** te geven, indien gewenst.

Maar **hoe beschrijven** we verschillende soorten van input op de tape (en dus als een **woord over een alfabet**)?

Definition

Voor een object (graaf, formule, getal) x duiden we met $\langle x \rangle$ de **representatie** van dat object aan als een woord.

Belangrijk is wel dat we in **polynomiale tijd** van x naar $\langle x \rangle$, en terug, kunnen gaan.

Voor een **geheel getal** n nemen we meestal $\langle n \rangle$ als de **binaire representatie** van dat getal.

- Een **unaire** representatie zou een slechte keuze zijn. Beschouw $\langle 10 \rangle = 1111111111$ versus $\langle 10 \rangle = 1010$.

Voor een **graaf** $G = (V, E)$ met knopen V en bogen $E \subseteq V \times V$, kan $\langle G \rangle$ bijvoorbeeld de beschrijving van G zijn als een lijst van bogen, of de beschrijving van G als een adjacency matrix.

- Voor een **gerichte** graaf $\langle G \rangle = \{(a, b), (b, c), (c, a)\}$ of

$$\langle G \rangle = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

of $\langle G \rangle = \{(a, b), (b, a), (b, c), (c, b), (a, c), (c, a)\}$ of

$$\langle G \rangle = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

voor een **ongerichte** graaf.

Het is meestal vanzelfsprekend hoe de input beschreven kan worden.

Veelvoorkomend probleem, bijvoorbeeld in navigatiesystemen,

- Dit vraagt echter exponentieel veel tijd: ruw afgeschat $\mathcal{O}(m^m)$, met m aantal knopen in G .

$M =$ Op input $\langle G, s, t \rangle$ met G een gerichte graaf met beginknoop s en eindknoop t :

1. Markeer de startknoop s . **(eenmaal)**
2. Herhaal hetvolgende totdat geen nieuwe markeringen op knopen gemaakt kunnen worden: **(m -keer)**
3. Doorloop de bogen in G . Als (a, b) een boog is met gemarkeerde knoop a en niet gemarkeerde knoop b , dan markeer b .
4. Als de eindknoop t gemarkeerd is, dan aanvaard de input, zoniet verwerp. **(eenmaal)**.

Stappen 1. en 4. kunnen duidelijk in polynomiale tijd. Stap 3. vraagt om de input te scannen en te testen welke knopen gemarkeerd zijn. Dit kan ook in polynomiale tijd. We herhalen stap 3. slechts m keer (m aantal knopen).

Propositie

PATH is in PTIME.

Definition

RELPRIME = $\{\langle x, y \rangle \mid x \text{ en } y \text{ zijn relatief priem}\}$.

Dit probleem is een subroutine gebruikt in **cryptografische protocollen** (bijv. RSA).

Hier, **relatief priem** betekent dat 1 de **enige gemeenschappelijke deler** is (10 en 21: ok, 10 en 22: niet ok).

Herinner: Getallen worden binair voorgesteld op de tape, dus, de lengte van $\langle x, y \rangle$ is $n = \mathcal{O}(\max\{\log_2(x), \log_2(y)\})$.

Naïeve oplossing: Probeer alle getallen tussen 2 en $\min\{x, y\}$ en check of deze x and y delen.

Merk op $\min\{x, y\}$ van orde grootte $\mathcal{O}(2^n)$ is met $n = \mathcal{O}(\max\{\log_2(x), \log_2(y)\})$. Dus dit naïef algoritme loopt in tijd **exponentieel** in n .

PTIME algoritme: Bereken de grootste gemene deler (gcd) door middel van het **algoritme van Euclides**.

R = Op input $\langle x, y \rangle$ met x en y in binaire representatie:

1. Herhaal totdat $y = 0$:
2. Laat $x \leftarrow x \bmod y$.
3. Vervang x met y (en y met x).
4. Als $x = 1$ aanvaard, anders verwerp.

Voorbeeld

$x = 10, y = 21 \mapsto x = 10 \bmod 21 = 10 \mapsto \text{swap} \mapsto x = 21, y = 10 \mapsto x = 21 \bmod 10 = 1 \mapsto \text{swap} \mapsto x = 1, y = 10 \mapsto x = 10 \bmod 1 = 0 \mapsto \text{swap } x = 1, y = 0$: aanvaard.

$x = 10, y = 22 \mapsto x = 10 \bmod 22 = 10 \mapsto \text{swap} \mapsto x = 22, y = 10 \mapsto x = 22 \bmod 10 = 2 \mapsto \text{swap} \mapsto x = 10, y = 2 \mapsto x = 10 \bmod 2 = 0 \mapsto \text{swap} \mapsto x = 2, y = 0$: verwerp.

Elke deler d van x en y is een deler van r_{k-1} .

$x = n_0 \cdot y + r_1$	$d \div x, d \div y, \text{ dan } d \div r_1$
$y = n_1 \cdot r_1 + r_2$	$d \div y, d \div r_1, \text{ dan } d \div r_2$
$r_1 = n_2 \cdot r_2 + r_3$	$d \div r_1, d \div r_2, \text{ dan } d \div r_3$
$\vdots$	$\vdots$
$r_{k-3} = n_{k-2} \cdot r_{k-2} + r_{k-1}$	$d \div r_{k-3}, d \div r_{k-2}, \text{ dan } d \div r_{k-1}$
$r_{k-2} = n_{k-1} \cdot r_{k-1} + r_k (= 0)$	$d \div r_{k-2} \text{ als en slechts dan } d \div r_{k-1}$

Is r_{k-1} ook een deler van x en y ?

$x = n_0 \cdot y + r_1$	$r_{k-1} \div r_1, r_{k-1} \div y, \text{ dan } r_{k-1} \div x$
$y = n_1 \cdot r_1 + r_2$	$r_{k-1} \div r_2, r_{k-1} \div r_1, \text{ dan } r_{k-1} \div y$
$r_1 = n_2 \cdot r_2 + r_3$	$r_{k-1} \div r_3, r_{k-1} \div r_2, \text{ dan } r_{k-1} \div r_1$
$\vdots$	$\vdots$
$r_{k-3} = n_{k-2} \cdot r_{k-2} + r_{k-1}$	$r_{k-1} \div r_{k-2}, r_{k-1} \div r_{k-1}, \text{ dan } r_{k-1} \div r_{k-3}$
$r_{k-2} = n_{k-1} \cdot r_{k-1} + r_k (= 0)$	$r_{k-1} \div r_{k-2}$

Inderdaad, $r_{k-1} = \gcd(x, y)$.

R = Op input $\langle x, y \rangle$ met x en y in binaire representatie:

1. Herhaal totdat $y = 0$:
2. Laat $x \leftarrow x \bmod y$.
3. Vervang x met y (en y met x).
4. Als $x = 1$ aanvaard, anders verwerp.

Complexiteit van Euclides' algoritme: (aantal iteraties) $\times$ (de tijd om $x \bmod y$ te berekenen).

- ▶ $x \bmod y$ is berekenbaar in polynomiale tijd.
- ▶ De stappen **1.–3.** worden slechts een **logaritmisch aantal keer** (in de grootte van de getallen) uitgevoerd (dus $\mathcal{O}(n)$ keer).

Tezamen levert dit een polynomiaal algoritme op.

R = Op input $\langle x, y \rangle$ met x en y in binaire representatie:

1. Herhaal totdat $y = 0$:
2. Laat $x \leftarrow x \bmod y$.
3. Vervang x met y (en y met x).
4. Als $x = 1$ aanvaard, anders verwerp.

Waarom loop het algoritme een **logaritmisch keer** in de grootte van de getallen?

- ▶ Na stap 2., $x < y$ omwille van modulo.
- ▶ Na stap 3., $x > y$ omwille van het verwisselen van x en y .
- ▶ In de volgende uitvoering van stap 2. is dus $x > y$. **Dan**,
 - ▶ als $x/2 \geq y \Rightarrow x \bmod y < y \leq x/2$, en dus x **halveert**, of
 - ▶ als $x/2 < y \Rightarrow x \bmod y = x - y < x/2$, en dus x **halveert**.

Omdat x en y verwisseld worden in stap 3., gaat ofwel x ofwel y halveren (afhankelijk van de iteratie)

Dit impliceert dat we stappen 2. & 3. maximaal $\min\{2\log_2(x), 2\log_2(y)\}$ keer uitvoeren.

Het is niet altijd **even eenvoudig** om aan te tonen of een probleem in PTIME zit:

Definition

PRIMES = $\{\langle x \rangle \mid x \text{ is a een priemgetal}\}$.

In 1801 vroeg de wiskundige Gauss zich af of het testen of een getal al dan niet priem is efficiënt kan worden beslist.

Pas in 2002 werd deze vraag positief beantwoord in een artikel

"PRIMES is in $P(\text{time})$ "

geschreven door drie Indiase onderzoekers Manindra Agrawal, Neeraj Kayal en Nitin Saxena.

Ze kregen hiervoor tal van prijzen en aandacht in de media...

Het algoritme hiervoor vraagt iets teveel voorkennis van getaltheorie om het te kunnen behandelen in deze cursus.

Deterministisch polynomiale tijd (PTIME)

Satisfiability problemen

Niet-deterministisch polynomiale tijd (NP)

Reductie, compleetheid en hardheid

De stelling van Cook-Levin

NP-complete problemen

Enorm veel problemen kunnen worden **herleid to satisfiability problemen**:

- ▶ Verificatie van circuits en software (Computer Aided Verification);
- ▶ Planning in Artificiële Intelligentie;
- ▶ Error diagnose van tal van systemen (bijv. chip design);
- ▶ En veel meer...

Specifieke toepassingen voor 2SAT:

- ▶ Labeling van diagrammen, scheduling van toernooien,...

en voor HORN-SAT:

- ▶ logica puzzels, analyse van discrete systemen,...

De verschillende satisfiability problemen komen later nog terug in de cursus.

Definition

Een **propositionele formule** φ is een uitdrukking die inductief gedefinieerd is als volgt,

- ▶ een (propositionele) **veranderlijke** in $X = \{x_1, x_2, \dots, x_k\}$ is een formule;
- ▶ als φ een formule is dan is ook $(\neg\varphi)$ een formule (**negatie**); en
- ▶ als φ_1 en φ_2 formules zijn, dan zijn ook $(\varphi_1 \wedge \varphi_2)$ (**conjunctie**) en $(\varphi_1 \vee \varphi_2)$ (**disjunctie**) formules.

Voorbeeld

$\Phi_1 = ((x_1 \wedge x_2) \vee x_3)$ en $\Phi_2 = ((\neg(((\neg x_1) \vee (\neg x_2)) \wedge (x_1 \vee x_3))) \vee (x_2 \vee (\neg x_3)))$ zijn propositionele formules.

Soms gebruiken we ook $\varphi_1 \rightarrow \varphi_2$ (**implicatie**) als notatie voor $((\neg\varphi_1) \vee \varphi_2)$.

Bijvoorbeeld, Φ_2 schrijven we ook als

$$(((\neg x_1) \vee (\neg x_2)) \wedge (x_1 \vee x_3)) \rightarrow (x_2 \vee (\neg x_3)).$$

Definition

Een **truth assignment** is een functie $\nu : X \rightarrow \{0, 1\}$ van veranderlijken in X naar 0 (onwaar) en 1 (waar).

Definition

Gegeven een propositionele formule φ over veranderlijken in X en een truth assignment $\nu : X \rightarrow \{0, 1\}$. We definiëren inductief wanneer ν de formule φ **waar maakt** (genoteerd als $\nu \models \varphi$):

- ▶ Als φ een veranderlijke $x \in X$ is, dan $\nu \models \varphi$ als en slechts dan $\nu(x) = 1$;
- ▶ Als $\varphi = (\neg\psi)$, dan $\nu \models \varphi$ als en slechts dan $\nu \not\models \psi$;
- ▶ Als $\varphi = (\psi_1 \wedge \psi_2)$, dan $\nu \models \varphi$ als en slechts dan $\nu \models \psi_1$ en $\nu \models \psi_2$; en
- ▶ Als $\varphi = (\psi_1 \vee \psi_2)$, dan $\nu \models \varphi$ als en slechts dan $\nu \models \psi_1$ of $\nu \models \psi_2$.

Voorbeeld

Stel $\nu(x_1) = 1$, $\nu(x_2) = 0$ en $\nu(x_3) = 0$, dan

$$\nu \not\models \Phi_1 \quad \text{en} \quad \nu \models \Phi_2,$$

met $\Phi_1 = ((x_1 \wedge x_2) \vee x_3)$ en $\Phi_2 = (((\neg x_1) \vee (\neg x_2)) \wedge (x_1 \vee x_3)) \rightarrow (x_2 \vee (\neg x_3))$.

Definition

Een propositionele formule φ is **satisfiable** als er een truth assignment ν over de veranderlijken in φ bestaat zodat $\nu \models \varphi$, en **unsatisfiable** als er geen truth assignment bestaat die φ waar maakt.

Definition

Twee propositionele formules φ en ψ over dezelfde veranderlijken zijn **equivalent**, genoteerd als $\varphi \equiv \psi$, als $\nu \models \varphi$ als en slechts dan $\nu \models \psi$, voor elke truth assignment ν van de veranderlijken.

Definition

Een propositionele formule φ over X is in **conjunctieve normaalvorm (CNF)** als die van de vorm^a $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_n$, is, waarin **clauses** C_i zijn, met elke clause van de vorm $C_i = \ell_1^i \vee \ell_2^i \vee \dots \vee \ell_{p_i}^i$, met daarin ℓ_j^i **literals**. Een literal is ofwel een veranderlijke (**positive literal** x) of een negatie van een veranderlijke (**negatieve literal** $\neg x$ of $\bar{x}$) in X .

^aWe laten even haakjes weg.

Theorem

Elke propositionele formule is equivalent met een propositionele formule in conjunctieve normaalvorm.

Bewijs: Druk negaties naar binnen door middel van de **wetten van De Morgan**:

$$(\neg(\varphi_1 \wedge \varphi_2)) \equiv ((\neg\varphi_1) \vee (\neg\varphi_2))$$

$$(\neg(\varphi_1 \vee \varphi_2)) \equiv ((\neg\varphi_1) \wedge (\neg\varphi_2))$$

$$(\neg(\neg\varphi)) \equiv (\varphi).$$

Vervolgens, **distribueer** $\vee$ over $\wedge$:

$$(\varphi_1 \vee (\varphi_2 \wedge \varphi_3)) \equiv ((\varphi_1 \vee \varphi_2) \wedge (\varphi_1 \vee \varphi_3)). \quad \square$$

Voorbeeld

De formule $\Phi_1 = ((x_1 \wedge x_2) \vee x_3)$ is equivalent met $((x_1 \vee x_3) \wedge (x_2 \vee x_3))$. Zet zelf Φ_2 om als oefening.

Soms leidt de omzetting naar CNF tot een **exponentiële groei**: $(x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee \dots \vee (x_k \wedge y_k)$ is equivalent met $(x_1 \vee x_2 \vee \dots \vee x_k) \wedge (y_1 \vee y_2 \vee \dots \vee y_k) \wedge (x_1 \vee y_2 \vee x_3 \vee \dots \vee x_k) \wedge \dots \wedge (y_1 \vee y_2 \vee x_3 \vee \dots \vee x_k) \wedge \dots \wedge (y_1 \vee y_2 \vee \dots \vee y_k)$ waarin elk van de k literals ofwel x_i of y_i kan zijn. Lengte is dus $\mathcal{O}(2^k)$.

Dit kan echter **vermeden worden** indien we **nieuwe** veranderlijken mogen gebruiken in de omzetting naar CNF.

Gegeven: een propositionele formule φ over veranderlijken in X . We maken nu $\text{cnf}(\varphi)$.

We **duwen eerst negatie door** tot op de veranderlijken, zoals voorheen:

$$\text{cnf}(\neg(\varphi_1 \vee \varphi_2)) = \text{cnf}((\neg\varphi_1) \wedge (\neg\varphi_2))$$

$$\text{cnf}(\neg(\varphi_1 \wedge \varphi_2)) = \text{cnf}((\neg\varphi_1) \vee (\neg\varphi_2))$$

$$\text{cnf}(\neg(\neg\varphi)) = \text{cnf}(\varphi)$$

voor **conjuncties** verandert er ook niets:

$$\text{cnf}(\varphi_1 \wedge \varphi_2) = \text{cnf}(\varphi_1) \wedge \text{cnf}(\varphi_2)$$

De verandering treedt op bij **disjuncties**.

$$\text{cnf}(\varphi_1 \vee \varphi_2) = (z \vee \text{cnf}(\varphi_1)) \wedge (\neg z \vee \text{cnf}(\varphi_2))$$

met z een **nieuwe** veranderlijke.

We distribueren vervolgens z and $\neg z$ over de clauses in $\text{cnf}(\varphi_1)$ en $\text{cnf}(\varphi_2)$, respectievelijk.

Voorbeeld

Stel $\varphi = (x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee (x_3 \wedge y_3)$. Dan,

$$\begin{aligned} \text{cnf}(\varphi) = & (z_1 \vee x_1) \wedge (z_1 \vee y_1) \wedge (\neg z_1 \vee z_2 \vee x_2) \wedge (\neg z_1 \vee z_2 \vee y_2) \wedge \\ & (\neg z_1 \vee \neg z_2 \vee x_3) \wedge (\neg z_1 \vee \neg z_2 \vee y_3). \end{aligned}$$

Dit resulteert duidelijk in een CNF formule, weliswaar over een grotere verzameling veranderlijken.

Propositie

Zij φ een propositionele formule over X and $\text{cnf}(\varphi)$ de CNF formule over X' met $X \subseteq X'$. Dan, voor elke truth assignment $\nu : X \rightarrow \{0, 1\}$ zodat $\nu \models \varphi$ bestaat er een truth assignment $\nu' : X' \rightarrow \{0, 1\}$ zodat $\nu' \models \text{cnf}(\varphi)$, en omgekeerd. Meer bepaald gaat $\nu'(x) = \nu(x)$ voor alle $x \in X$.

Bewijs: De geldigheid van deze propositie is eenvoudig in te zien per inductie. We mogen ervan uitgaan dat negatie is doorgedrukt tot op de veranderlijken.

Ook is de rest van de transformatie naar CNF te zien als de toepassing van de regel

$$(\varphi_1 \wedge \varphi_2) \vee \varphi_3 \mapsto (z \vee \varphi_1) \wedge (z \vee \varphi_2) \wedge (\neg z \vee \varphi_3).$$

Veronderstel nu dat $(\varphi_1 \wedge \varphi_2) \vee \varphi_3$ satisfiable is en zij ν een truth assignment zodat $\nu \models (\varphi_1 \wedge \varphi_2) \vee \varphi_3$

- ▶ Veronderstel dat $\nu \models \varphi_1 \wedge \varphi_2$. Dan definiëren we $\nu'(z) = 0$ en $\nu'(x) = \nu(x)$ voor alle andere veranderlijken.
- ▶ Veronderstel dat $\nu \models \varphi_3$. Dan definiëren we $\nu'(z) = 1$ en $\nu'(x) = \nu(x)$ voor alle andere veranderlijken.

Dan, $\nu' \models (z \vee \varphi_1) \wedge (z \vee \varphi_2) \wedge (\neg z \vee \varphi_3)$.

Omgekeerd, veronderstel dat $(z \vee \varphi_1) \wedge (z \vee \varphi_2) \wedge (\neg z \vee \varphi_3)$ satisfiable is en zij ν' een truth assignment die deze formule waar maakt.

- ▶ Veronderstel dat $\nu'(z) = 1$. Dan moet $\nu' \models \varphi_3$ en dus gaat ook de restrictie van ν' op de veranderlijken in de oorspronkelijke formule φ_3 waar maken.
- ▶ Veronderstel dat $\nu'(z) = 0$. Dan moet $\nu' \models \varphi_1 \wedge \varphi_2$ en dus gaat ook de restrictie van ν' op de veranderlijken in de oorspronkelijke formule $\varphi_1 \wedge \varphi_2$ waar maken.

Dan, $\nu \models (\varphi_1 \wedge \varphi_2) \vee \varphi_3$.

Is $\text{cnf}(\varphi)$ van polynomiale lengte in $|\varphi|$?

In elke stap komen er een **constant aantal** symbolen bij. □

In de normale CNF transformatie **verdubbelen we de lengte** tijdens het distribueren van $\vee$ over $\wedge$.

Definition

SAT = $\{\langle \varphi \rangle \mid \varphi \text{ is een satisfiable propositionele formule}\}$.

CNF-SAT = $\{\langle \varphi \rangle \mid \varphi \text{ is een satisfiable propositionele formule in CNF}\}$.

3SAT = $\{\langle \varphi \rangle \mid \varphi \text{ is een satisfiable propositionele formule in CNF}$
waarin elke clause 3 literals heeft $\}$.

2SAT = $\{\langle \varphi \rangle \mid \varphi \text{ is een satisfiable propositionele formule in CNF}$
waarin elke clause 2 literals heeft $\}$.

HORN-SAT = $\{\langle \varphi \rangle \mid \varphi \text{ is een satisfiable propositionele formule in CNF}$
waarin elke clause maximaal één positive literal heeft $\}$.

Speciaal aan clauses van de vorm $(\ell \vee \ell')$ met ℓ en ℓ' literals $x, y, \neg x$ of $\neg y$, dat elke truth assignment $\nu : \{x, y\} \rightarrow \{0, 1\}$ zodat $\nu \models (\ell \vee \ell')$ **moet** voldoen aan: als $\nu(\ell) = 0$ dan $\nu(\ell') = 1$ en omgekeerd, als $\nu(\ell') = 0$ dan $\nu(\ell) = 1$.

Voorbeeld

Beschouw

$$\varphi = (x_1 \vee x_2)(x_1 \vee x_2) \wedge (x_1 \vee \neg x_3)(x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2)(\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)(x_2 \vee x_3)$$

$$\neg x_1 \rightarrow x_2$$

$$\neg x_2 \rightarrow x_1$$

$$\neg x_1 \rightarrow \neg x_3$$

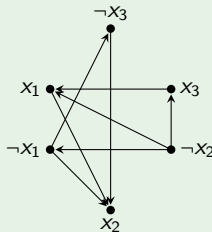
$$x_3 \rightarrow x_1$$

$$x_1 \rightarrow x_2$$

$$\neg x_2 \rightarrow \neg x_1$$

$$\neg x_2 \rightarrow x_3$$

$$\neg x_3 \rightarrow x_2$$



Dan, φ is satisfiable als en slechts dan er geen enkel pad bestaat tussen knopen gelabeld met x en $\neg x$, en omgekeerd, en dit voor elke veranderlijke x in φ .

Intuïtie: zo'n betekent dat zowel x als $\neg x$ waar moeten zijn.

Dergelijke **implicatie graaf** kan in polynomiale tijd berekend worden, en zo ook

$M =$ Op input $\langle \varphi \rangle$ met φ CNF met clauses bestaande uit 2 variabelen:

1. Maak knopen x en $\neg x$ voor elke veranderlijke in φ .
2. Maak gerichte bogen $(\neg \ell, \ell')$ en $(\neg \ell', \ell)$ als $\ell \vee \ell'$ een clause is in φ . Dit resulteert in de implicatie graaf G van φ .
3. Herhaal hetvolgende voor elke veranderlijke x in φ :
4. Is $\langle G, x, \neg x \rangle \in \text{PATH}$ en $\langle G, \neg x, x \rangle \in \text{PATH}$?
5. Als het antwoord steeds negatief is, dan aanvaard. Anders, verwerp.

Dit kan duidelijk allemaal in polynomiale tijd.

Theorem

2SAT is in PTIME.

Waarom werkt dit algoritme?

We tonen aan dat als er een pad van een veranderlijke x naar $\neg x$ is, en terug, dat φ dan niet satisfiable is.

Stel, er is een veranderlijke x met een pad $P_{x \mapsto \neg x}$ naar $\neg x$ en een pad $P_{\neg x \mapsto x}$ terug naar x in de implicatie graaf van φ .

Als we nu $\nu(x) = 1$, dan moet elke literal in $P_{x \mapsto \neg x}$ op “waar” gezet worden. Echter, $\nu(\neg x)$ moet op nul gezet worden.

Als we nu $\nu(x) = 0$, dan moet $\nu(\neg x) = 1$ zijn. Opnieuw moet elke literal $P_{\neg x \mapsto x}$ op “waar” gezet worden. Echter, $\nu(x)$ was reeds op 0 gezet.

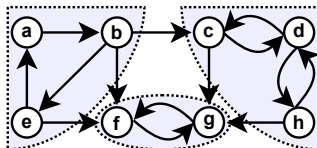
Er mogen dus geen paden zijn van x naar $\neg x$ en terug.

Zij $G = (V, E)$ een gerichte graaf.

- ▶ We noemen G **sterk samenhangend** als er tussen elke twee knopen in G een pad bestaat.
- ▶ Voor elke deelverzameling V' van knopen in V kunnen we de **deelgraaf** $G[V'] = (V', E')$ bekijken, waarin

$$E' := \{(u, v) \mid (u, v) \in E, u, v \in V'\}$$

- ▶ Een **sterk samenhangende component** C van G is een sterk samenhangende **deelgraaf** van de vorm $G[V']$, met V' maximaal. Dat is, $G[V' \cup \{v\}]$ is niet meer sterk samenhangend voor elke knoop v niet in V' .



Omgekeerd, stel dat er geen zo'n veranderlijken bestaan dan zal φ satisfiable zijn.

Zij $C(x)$ de verzameling van knopen in de sterk samenhangende component (s.s.c) van x in G . Dat is,

$$C(x) = \{y \mid \text{alle literals } y \text{ die bereikt kunnen worden met een pad vanuit } x \\ \text{en waarvoor er een pad terug bestaat van } y \text{ naar } x.\}$$

We maken nu een nieuwe graaf:

- ▶ Voor elke s.s.c. C in G , maak een nieuwe knoop v_C . Noteer deze verzameling knopen met V_{SSC} .
- ▶ Voor elke paar knopen v_C en $v_{C'}$, voeg een boog $(v_C, v_{C'})$ toe indien er een boog was in G vanuit een knoop in C naar C' . Noteer deze verzameling bogen met E_{SSC} .

De graaf $G_{SSC} = (V_{SSC}, E_{SSC})$ is een gerichte **acyclische** graaf (DAG).

Er bestaat een **topologische sortering** van de knopen in G_{SSC} .

Met andere woorden, we kunnen de knopen in V_{SSC} nummeren met een functie $i : V_{SSC} \rightarrow \mathbb{N}$ zodat als $(v_C, v_{C'})$ een boog is dan moet $i(v_C) < i(v_{C'})$.

Merk op dat s.s.c van x and de s.s.c van $\neg x$ verschillend zijn. Dit volgt uit onze aanname.

We definiëren nu de truth assignment $\nu : X \rightarrow \{0, 1\}$:

$$\nu(x) = \begin{cases} 1 & \text{als } i(v_{C(\neg x)}) < i(v_{C(x)}) \\ 0 & \text{anders.} \end{cases}$$

Veronderstel nu dat $\nu \not\models \varphi$. We gaan dan tot een contradictie komen. En dus $\nu \models \varphi$.

Uit $\nu \not\models \varphi$ volgt dat er een clause $(\ell \vee \ell')$ in φ bestaat zodat $\nu \not\models (\ell \vee \ell')$.

Met andere woorden, $\nu(\ell) = 0$ en $\nu(\ell') = 0$.

Herinner:

$$\nu(x) = \begin{cases} 1 & \text{als } i(\nu_{C(\neg x)}) < i(\nu_{C(x)}) \\ 0 & \text{als } i(\nu_{C(\neg x)}) \geq i(\nu_{C(x)}) \end{cases}$$

en $\nu(\ell) = 0$ en $\nu(\ell') = 0$.

Dit wil zeggen dat

$$i(\nu_{C(\neg \ell)}) \geq i(\nu_{C(\ell)}) \text{ en } i(\nu_{C(\neg \ell')}) \geq i(\nu_{C(\ell')}).$$

Merk nu op dat er bogen zijn van $\neg \ell$ naar ℓ' en van $\neg \ell'$ naar ℓ .

Echter, dit impliceert dat

$$i(\nu_{C(\neg \ell)}) \leq i(\nu_{C(\ell')}) \text{ en } i(\nu_{C(\neg \ell')}) \leq i(\nu_{C(\ell)})$$

en dus

$$i(\nu_{C(\neg \ell)}) = i(\nu_{C(\ell')}) = i(\nu_{C(\neg \ell')}) = i(\nu_{C(\ell)}),$$

wat niet kan.



Voorbeeld

- ▶ Als Mhr. Smets een hond heeft, dan heeft Mevr. Baeten een kat.
- ▶ Als Mhr. Jansen een hond heeft, dan heeft hij ook een kat.
- ▶ Als Mhr. Smets een hond heeft en Mhr. Jansen een kat, dan heeft Mevr. Paulussen een hond.
- ▶ Als Mevr. Baeten en Mhr. Jansen een huisdier hebben van dezelfde soort, dan heeft Mhr. Smets een kat.
- ▶ Alle mannen hebben honden.

Vraag: Wie heeft wat voor huisdieren?

We kunnen dergelijke “kennis” encoderen in propositionele formules:

- ▶ variabelen: $s(\text{mets})$, $j(\text{ansen})$, $b(\text{aeten})$, $p(\text{aulussen})$.
- ▶ waarden: true, false afhankelijk van bezit van hond.
- ▶ variabelen: $S(\text{mets})$, $J(\text{ansen})$, $B(\text{aeten})$, $P(\text{aulussen})$.
- ▶ waarden: true, false afhankelijk van bezit van kat.

Voorbeeld

De veronderstelling in het voorbeeld kunnen dan worden geschreven als een **Horn formule** (CNF met maximaal één positieve literal per clause):

- ▶ $\varphi_1 = s \rightarrow B$
- ▶ $\varphi_2 = j \rightarrow J$
- ▶ $\varphi_3 = (s \wedge J) \rightarrow p$
- ▶ $\varphi_4 = (b \wedge j \rightarrow S) \wedge (B \wedge J \rightarrow S)$
- ▶ $\varphi_5 = s \wedge j.$

Dan, alle truth assignments voor s, j, b, p, S, J, B, P die voldoen aan

$$\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4 \wedge \varphi_5$$

of, equivalent voor (herinner: $p \rightarrow q \equiv \neg p \vee q$)

$(\neg s \vee B) \wedge (\neg j \vee J) \wedge (\neg s \vee \neg J \vee p) \wedge (\neg b \vee \neg j \vee S) \wedge (\neg B \vee \neg J \vee S) \wedge s \wedge j$
zijn oplossingen op de vraag "Wie heeft wat voor huisdieren?" .

Wat is er speciaal aan Horn formules?

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

met elke clause C_i van de vorm:

- ▶ x of $\neg x$;
- ▶ $\neg x_1 \vee \cdots \vee \neg x_k$ of $\neg x_1 \vee \cdots \vee \neg x_k \vee x$

Lemma

Als elke clause in φ een negatieve literal bevat dan is φ satisfiable.

Bewijs. Kies voor elke C_i een negatieve literal. Stop de bijhorende variabelen in verzameling X' .

Definieër: $\nu(x) = 0$ voor alle $x \in X'$, $\nu(x) = 0/1$ voor alle $x \notin X'$.

Het is duidelijk dat $\nu \models C_i$ voor alle $i = 1, \dots, m$.

En dus ook $\nu \models \varphi$.



Meer algemeen, neem een Horn formule φ met een negatieve literal in elke clause. Zij ψ een conjunctie van literals (positief en negatief). Veronderstel dat de bijhorende variabelen in ψ niet in φ voorkomen.

Voorbeeld

Neem $\varphi = (\neg a \vee \neg b \vee c) \wedge (\neg a \vee \neg c)$ en $\psi = d \wedge \neg e \wedge f$.

Propositie

Neem φ en ψ zoals hierboven. De Horn formule $\Phi = \varphi \wedge \psi$ is satisfiable als en slechts dan ψ satisfiable is. Meer nog, Φ is satisfiable als en slechts dan in ψ niet zowel x als de negatie $\neg x$ voorkomt, voor een veranderlijke x .

Bewijs. We weten all dat φ satisfiable is. Aangezien φ en ψ over verschillende veranderlijken gedefinieerd zijn is dus Φ satisfiable als en slechts dan ψ satisfiable is.

Aangezien ψ een conjunctie is van literals kan ψ alleen maar satisfiable zijn als er geen enkele veranderlijke tesamen met zijn negatie in voorkomt. $\square$

We noemen een clause bestaande uit een literal een **unit clause**.

We gaan dit nu gebruiken om een algoritme te maken voor HORN-SAT.
Gegeven een Horn formule Φ .

1. Zolang er nog (nieuwe) unit clauses zijn in Φ :
2. Selecteer een (nieuwe) unit clause, bijv p en maak die waar (true)
3. Verwijder elke clause (behalve unit clauses) die p bevat (ok, omdat $(p \vee X) \equiv \text{true}$)
4. Verwijder $\neg p$ van elke (niet unit) clause (ok, omdat $\neg p \vee X \equiv X$)
5. Als we unit clauses p and $\neg p$ hebben, verwerp.
6. Anders, aanvaard.

Waarom is dit juist?

Schrijf $\Phi = \varphi \wedge \psi$, waarbij ψ een maximale conjunctie is van unit clauses in Φ .

Als ψ leeg is, dan zijn er geen unit clauses. Dan, φ is satisfiable (zie lemma) en stap 6. wordt direkt uitgevoerd. Φ wordt dus correct aanvaard.

Als ψ niet leeg is dan, omdat Φ is satisfiable als en slechts dan φ en ψ satisfiable zijn, zal stap 5. de initiële input verwerpen als ψ (en dus Φ) niet satisfiable is.

Echter, stel dat stap 5. de initiële input Φ niet verworpt, dan betekent dit niet dat Φ satisfiable is.

Het algoritme gaat nu φ en ψ zo transformeren dat deze voldoen aan de voorwaarden van het vorige lemma.

Meer bepaald, stel dat $\Phi \equiv \varphi^{(i-1)} \wedge \psi^{(i-1)}$ net voor de i de iteratie.

Er is dus unit clause p (resp. $\neg p$) in $\psi^{(i-1)}$ die nog niet behandeld werd.

Het algoritme gaat dan:

- ▶ Elke clause in $\varphi^{(i-1)}$ elimineren waar p in zit;
- ▶ Elke clause $C = \neg p \vee D$ in $\varphi^{(i-1)}$ vervangen door D .
- ▶ En als dergelijke D 's unit clauses geworden zijn, dan worden die aan $\psi^{(i-1)}$ toegevoegd.

Noteer de bekomen formules met $\varphi^{(i)}$ en $\psi^{(i)}$. We hebben: $\Phi \equiv \varphi^{(i)} \wedge \psi^{(i)}$.

Merk op dat $\varphi^{(i)}$ en $\psi^{(i)}$ een gemeenschappelijke veranderlijke p minder hebben.

De juistheid van stap 5. met betrekking tot $\psi^{(i)}$ volgt uit dezelfde redenering als voordien.

Merk nu op dat het algoritme stopt, stel in stap k als $\Phi \equiv \varphi^{(k)} \wedge \psi^{(i)}$ zodra $\varphi^{(k)}$ geen unit clauses meer bevat en de veranderlijken in de literals van $\psi^{(i)}$ niet voorkomen in $\varphi^{(k)}$.

Dus, $\varphi^{(k)}$ is steeds satisfiable en Φ is satisfiable als en slechts dan stap 5. $\psi^{(k)}$ niet verworpt. $\square$

Een truth assignment voor Φ valt nu af te lezen uit $\psi^{(k)}$.

Het algoritme loopt in $\mathcal{O}(mn)$ met m het aantal clauses en n de lengte van de formule.

Dit kan verminderd worden naar $\mathcal{O}(n)$ (speciale data-structuren).

Voorbeeld

We starten met

$$(\neg s \vee B) \wedge (\neg j \vee J) \wedge (\neg s \vee \neg J \vee p) \wedge (\neg b \vee \neg j \vee S) \wedge (\neg B \vee \neg J \vee S) \wedge s \wedge j.$$

Kies een **unit clause** (literal) en bijbehorende truth value, bijv. j and $j \mapsto \text{true}$.

Simplificeer formule $((j \vee \varphi \equiv \text{true}), (\neg j \vee \varphi \equiv \varphi))$

$$(\neg s \vee B) \wedge J \wedge (\neg s \vee \neg J \vee p) \wedge (\neg b \vee S) \wedge (\neg B \vee \neg J \vee S) \wedge s \wedge j,$$

en kies opnieuw een unit clause en bijbehorende truth value, bijv. s and $s \mapsto \text{true}$.

Simplificeer formule.

$$B \wedge J \wedge (\neg J \vee p) \wedge (\neg b \vee S) \wedge (\neg B \vee \neg J \vee S) \wedge s \wedge j$$

en zo gaan we door...

Voorbeeld

$$B \wedge J \wedge (\neg J \vee p) \wedge (\neg b \vee S) \wedge (\neg B \vee \neg J \vee S) \wedge s \wedge j$$

kies unit clause en bijbehorende truth value, bijv. B and $B \mapsto \text{true}$. Simplificeer formule.

$$B \wedge J \wedge (\neg J \vee p) \wedge (\neg b \vee S) \wedge (\neg J \vee S) \wedge s \wedge j$$

kies unit clause en bijbehorende truth value, bijv. J and $J \mapsto \text{true}$. Simplificeer formule.

$$B \wedge J \wedge p \wedge (\neg b \vee S) \wedge S \wedge s \wedge j$$

kies unit clause en bijbehorende truth value, bijv. S and $S \mapsto \text{true}$. Simplificeer formule.

$$B \wedge J \wedge p \wedge S \wedge s \wedge j$$

Conclusie: Mhr. Smets en Jansen hebben hond en kat, Mevr. Baeten heeft een kat (en misschien een hond), Mevr. Paulussen heeft een hond (en misschien een kat).

Deterministisch polynomiale tijd (PTIME)

Satisfiability problemen

Niet-deterministisch polynomiale tijd (NP)

Reductie, compleetheid en hardheid

De stelling van Cook-Levin

NP-complete problemen

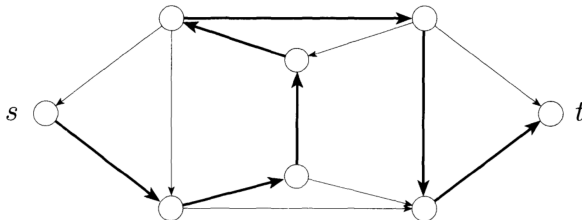
PTIME is efficiënt, maar wat als een probleem **intrinsiek moeilijk** is (bijvoorbeeld wanneer we slechts een exponentieel algoritme ervoor kennen)

Bijvoorbeeld:

Definition

HAMPATH = $\{ \langle G, s, t \rangle \mid G \text{ is een gerichte graaf met een} \\ \text{Hamiltoniaans pad van } s \text{ naar } t \}.$

(**Hamiltoniaans pad** = pad dat **elke** knoop exact **éénmaal** bezoekt)



Naïef algoritme: Genereer alle paden van lengte n (aantal knopen) en check of het een Hamiltoniaans pad is. (aantal paden is exponentieel in grootte van input graph).

We merken op:

- ▶ We kunnen **testen** of een **gegeven** pad Hamiltoniaans is in **polynomiale tijd**.
- ▶ Maar we weten niet hoe snel en direct dergelijk pad **te vinden**.

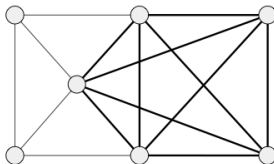
Het verifiëren of een pad Hamiltoniaans is, is eenvoudiger dan het beslissen of er zo één is.

Een **gegeven** pad = **getuige (certificaat, bewijs)** voor het al dan niet Hamiltoniaans zijn.

Dergelijke situatie komt veel voor:

Voorbeeld

- ▶ SAT: gegeven een truth assignment is het gemakkelijk om te testen of de formule waar is, het vinden van zo'n truth assignment is echter niet zo eenvoudig.
- ▶ Idem voor CNF-SAT en 3SAT.



Definition

- ▶ Een **clique** in een ongerichte graaf is een deelgraaf waarin elke twee knopen verbonden zijn met een boog.
- ▶ Een **k -clique** is een clique bestaande uit k knopen.

Het vinden van cliques is een belangrijk probleem in het analyseren van social networks.

Definition

CLIQUE = $\{ \langle G, k \rangle \mid G \text{ is een ongerichte graaf met een } k\text{-clique} \}$.

Voorbeeld

- ▶ **CLIQUE**: gegeven k knopen is het gemakkelijk om de testen of deze een clique vormen, het vinden van dergelijke k knopen niet zo eenvoudig.

Definition

- ▶ Een **verifiër** voor een taal L is een deterministische Turing machine V zodat $L = \{w \mid V \text{ aanvaardt } \langle w, c \rangle \text{ voor een woord } c\}$.
- ▶ Een **polynomiale tijd verifiër** V loopt in polynomiale tijd in $|w|$.
- ▶ Een taal is **polynomiaal verifieerbaar** als het een polynomiale tijd verifiër heeft.

Het woord c = getuige, certificaat voor het “lidmaatschap” (element zijn van) de taal L

Voor **polynomiale tijd verifiers** is c van **polynomiale lengte** in $|w|$.

Voorbeeld

- ▶ HAMPATH: c is een Hamiltoniaans pad.
- ▶ SAT: c is een truth assignment die de formule waar maakt.
- ▶ CLIQUE: c is een k -clique.

Omdat polynomiale verifieerbaarheid zo fundamenteel en veelvoorkomend is heeft de verzameling van talen met deze eigenschappen een eigen naam:

Definition

NP is de klasse van alle talen die polynomiale verifiers hebben.

Deze klasse bevat veel (bijna alle) problemen die relevant zijn in toepassingen,

De naam NP komt eigenlijk van Niet-deterministisch Polynomiale tijd...

Herinner,

Definition

$\text{CLIQUE} = \{ \langle G, k \rangle \mid G \text{ is een ongerichte graaf met een } k\text{-clique} \}.$

Een **polynomiale verifiër** voor CLIQUE:

$V = \text{Op input } \langle \langle G, k \rangle, c \rangle:$

1. Check dat c uit k knopen van G bestaat;
2. Check dat G bogen heeft tussen elke twee knopen in c ;
3. Aanvaard als beide testen positief zijn, anders verwerp.

Alternatief: Gebruik een **niet-deterministische TM**:

$M =$ Op input $\langle G, k \rangle$:

1. Kies, op een *niet-deterministische* manier k knopen van G en stop die een set verzameling c ;
2. Test of dat c een k -clique is in G ;
3. Als dat zo is, aanvaard, anders verwerp.

Algemene strategie: Polynomiale “gok”, gevolgd door een polynomiale check.

Dergelijke gok komt overeen met het volgen van één van de takken in een berekeningsboom van de NTM.

Definition

$\text{HAMPATH} = \{ \langle G, s, t \rangle \mid G \text{ is een gerichte graaf met een Hamiltoniaans pad van } s \text{ naar } t \}.$

Ook beslisbaar door NTM:

$N = \text{Op input } \langle G, s, t \rangle$, met G een gerichte graaf met knopen s en t :

1. Kies op een niet-deterministische manier een lijst $p_1, \dots, p_n$ van knopen in G , met n het aantal knopen in G ;
2. Check ofdat deze lijst dubbels bevat. Als dit zo is, verwerp;
3. Check ofdat $s = p_1$ en $t = p_n$. Zoniet, verwerp;
4. Check ofdat elk paar (p_i, p_{i+1}) een boog is in G . Zoniet verwerp, anders aanvaard.

Herinner dat de runtime van een NTM (beslisser) de lengte is van de langste tak in de berekeningsboom.

Dit is duidelijk een NTM dewelke in polynomiale tijd loopt.

Definition

$\text{SAT} = \{ \langle \varphi \rangle \mid \varphi \text{ is een satisfiable propositionele formule} \}.$

Ook beslisbaar door NTM:

$N = \text{Op input } \langle \varphi \rangle, \text{ met } \varphi \text{ een propositionele formule in de veranderlijken } X = \{x_1, \dots, x_n\}:$

1. Kies op een niet-deterministische manier een truth assignment $\nu : X \rightarrow \{0, 1\}$;
2. Check dat deze truth assignment φ waar maakt ($\nu \models \varphi$). Zoniet, verwerp, anders aanvaard.

Het verband tussen polynomiale verifiëerbaarheid en beslisbaarheid door een NTM is niet toevallig...

Theorem

Een taal is NP (heeft een polynomiale verifiër) als en slechts dan deze taal beslist wordt door een niet-deterministische TM die loopt in polynomiale tijd.

Bewijs: Neem een taal $L \in \text{NP}$ en stel dat V een polynomiale verifiër is voor L . Herinner, V is een deterministische TM die in polynomiale tijd loopt. Met andere woorden, V doet $\mathcal{O}(n^k)$ stappen voor een zeker k op inputs van grootte n .

Er bestaat dus een constante d zodat V $d \cdot n^k$ stappen zet op een input van grootte n .

We definiëren nu de volgende NTM:

N = Op input w van lengte n :

1. Kies op een niet-deterministische manier een woord c van lengte $\leq d \cdot n^k$.
2. Voer V uit op input $\langle w, c \rangle$.
3. Als V deze input aanvaard, aanvaard, anders verwerp.

Omgekeerd: Stel dat N een NTM is die de taal L beslist in polynomiale tijd. De bijhorende verifiër V werkt dan als volgt:

$V =$ Op input $\langle w, c \rangle$ met woorden w and c :

1. Simuleer de run van N op input w waarbij c de beschrijving (keuzes) is van één van de takken in N 's berekeningsboom
2. Als deze tak w aanvaard, aanvaard, anders verwerp. □

Het is soms eenvoudiger om te werken met de volgende definitie van NP:

Definition

$$\text{NP} = \bigcup_k \text{NTIME}(n^k).$$

Waarin, we net zoals voor deterministische TMs:

Definition

$$\text{NTIME}(t(n)) = \{L \mid L \text{ is een taal die beslist wordt door een} \\ \text{niet-deterministische Turing machine in tijd } \mathcal{O}(t(n)) \}.$$

Informeel:

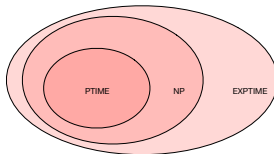
- ▶ PTIME = talen waarvoor lidmaatschap snel beslist kan worden.
- ▶ NP = talen waarvoor lidmaatschap snel geverifieerd kan worden.

Maar hoe lossen we nu NP problemen op een echte computer?

- ▶ Hierop kennen we het antwoord al: Een polynomiale tijd NTM kan worden omgevormd in een TM die exponentiële tijd vraagt.

Theorem

$$PTIME \subseteq NP \subseteq EXPTIME = \bigcup_k DTIME(2^{n^k})$$



Million dollar question

Is $PTIME = NP$?

Dit is niet geweten. Als het niet zo is, dan zijn moeilijke problemen slechts oplosbaar door brute-force search. Als het wel zo is, dan zijn alle praktische

- ▶ zoekproblemen,
- ▶ optimalisatieproblemen,
- ▶ decryptieproblemen, en
- ▶ machine learning en AI problemen

in polynomiale tijd berekenbaar.

Daarom is het één van de zeven “Millennium Prize Problems”. Het oplossen hiervan = 1 miljoen US \$.

Deterministisch polynomiale tijd (PTIME)

Satisfiability problemen

Niet-deterministisch polynomiale tijd (NP)

Reductie, compleetheid en hardheid

De stelling van Cook-Levin

NP-complete problemen

Als een probleem in NP zit betekent dit nog niet dat er geen PTIME algoritme voor bestaat (zelfs als zou blijken dat $\text{PTIME} \neq \text{NP}$),

De vraag is dus **hoe we zeker kunnen zijn dat een probleem intrinsiek moeilijk is.**

Goede poging: Toon aan dat je probleem even moeilijk is om op te lossen als **om het even** ander probleem in NP.

Dergelijke problemen worden dan **NP-hard** genoemd.

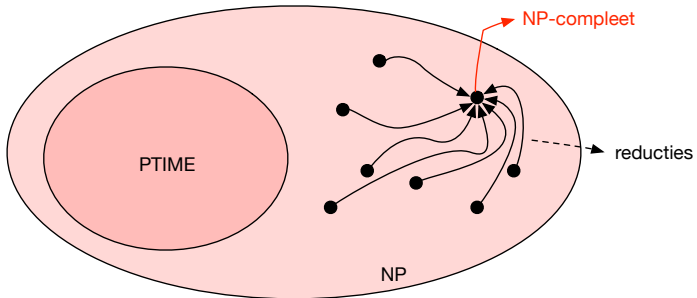
Ook van belang is om problemen te identificeren die een complexiteitsklasse **representeren**.

Met andere woorden, wanneer het oplossen ervan **all problemen** in die klasse oplost.

Dergelijke problemen worden dan **compleet** genoemd t.o.v. een complexiteitsklasse, bijvoorbeeld **NP-compleet** voor de klasse NP.

Informeel: Een NP-compleet probleem zit in NP en is NP-hard.

We gaan dat nu formaliseren.

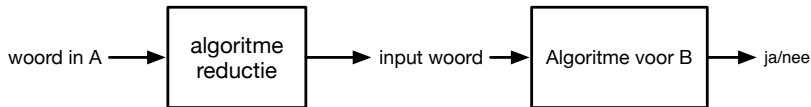


Om hardheid en compleetheid te definiëren hebben we formele manier nodig om het ene probleem te transformeren (**reduceren**) naar een ander probleem.

We moeten **een algoritme maken** dat elke instantie van een probleem A omzet in een instantie van een probleem B , en dit op een efficiënte manier.

Zo'n algoritme lost dus geen beslissingsprobleem maar genereert een output.

Dergelijke algoritmen worden gemodelleerd door **TM die functies berekenen**.



Definition

Een functie $f : \Sigma^* \rightarrow \Sigma^*$ is een **polynomiale tijd berekenbare functie** als er een polynomiale tijd TM bestaat die steeds stopt en op input w , na het stoppen slechts $f(w)$ op de tape heeft staan.

Definition

Een taal A is **polynomiale tijd reduceerbaar** naar een taal B , genoteerd als $A \leq_p B$, als er een polynomiale tijd berekenbare functie $f : \Sigma^* \rightarrow \Sigma^*$ bestaat zodat

$$w \in A \iff f(w) \in B.$$

We zeggen ook dat A **polynomiaal reduceerbaar** is naar B . De functie f wordt de **reductie** genoemd van A naar B .

Als $A \leq_p B$ dan kunnen we dus algoritmen gebruiken die B beslissen om ook A te beslissen.

Theorem

3SAT is polynomiale tijd reduceerbaar naar CLIQUE.

Gevolg: Elk algoritme voor CLIQUE kan gebruikt worden voor 3SAT op te lossen.

Merk op dat een reductie in één richting werkt!

Stel, we nemen een instantie

$$\varphi = \underbrace{(a_1 \vee b_1 \vee c_1)}_{C_1} \wedge \cdots \wedge \underbrace{(a_k \vee b_k \vee c_k)}_{C_k}.$$

van 3SAT. We moeten nu een efficient algoritme maken dat de representatie $\langle \varphi \rangle$ van φ omzet in $\langle G, k \rangle$, een instantie van CLIQUE.

Een mogelijke reductie $f : \Sigma^* \rightarrow \Sigma^*$ is als volgt:

Voor de knopen:

- ▶ G heeft k groepjes, elk groepje bestaande uit 3 knopen.
- ▶ Elk groepje gaan we overeen laten komen met een clause C_i in φ .
- ▶ De knopen in een groepje zijn **gelabeld met de literals in de corresponderende clause**.

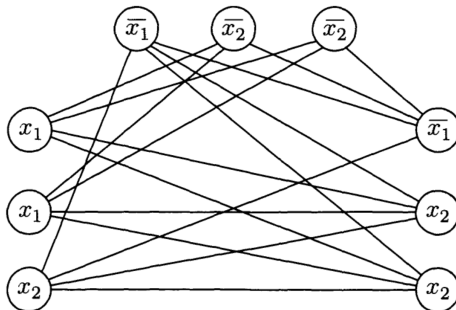
Voor de bogen:

- ▶ Er zijn geen bogen tussen knopen in **hetzelfde** groepje.
- ▶ Er zijn geen bogen tussen knopen die **gelabeld met x en de negatie $\bar{x}$** .
- ▶ Er zijn wel bogen tussen alle andere knopen.

De reductie f is duidelijk een polynomiaal berekenbare functie.

Voorbeeld

$$\varphi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$



Is dit voldoende? Is $w \in 3SAT \iff f(w) \in CLIQUE$?

$\Rightarrow$ Stel dat φ satisfiable is. M.a.w., er is een truth assignment $\nu : X \rightarrow \{0, 1\}$ met X de veranderlijken in φ zodat $\nu \models \varphi$.

In elk van de k groepjes in G is er dus een knoop die overeenkomt met een literal die op true wordt gezet door ν . Neem uit elk groepje één zo'n knoop.

Definiëer de deelgraaf C van G bestaande uit deze k knopen (en dus ook de bogen in G tussen deze k knopen)

Claim

C is een k -clique.

Bewijs: de enige knopen waar geen boog tussen is in zijn:

- ▶ knopen in dezelfde groep, echter in C zitten k knopen uit verschillende groepjes,
- ▶ knopen die gelabeld zijn met x en $\bar{x}$, echter de truth assignment ν maakt ofwel x ofwel $\bar{x}$ waar, maar nooit beiden.

En dus C is een k -clique



$\Leftarrow$ Stel, G heeft een k -clique C .

Aangezien er geen bogen zijn tussen knopen in eenzelfde groep, bevat C één enkele knoop per groep.

Stel dat $\ell_1, \dots, \ell_k$ de literals zijn die deze k knopen labelen. Merk op dat als $\ell_i = x$ dan is geen enkele $\ell_j = \bar{x}$, en omgekeerd. (Waarom? Anders zou er geen boog zijn tussen de corresponderende knopen.)

Verzamel alle veranderlijken $X' \subseteq X$ die voorkomen in $\ell_1, \dots, \ell_k$.

Definieer voor $x \in X'$:

$$\nu(x) = \begin{cases} 1 & \text{als } \ell_i = x \text{ voor een zekere } i \\ 0 & \text{als } \ell_i = \bar{x} \text{ voor een zekere } i \end{cases}$$

Verder, breid ν uit tot een truth assignment op de volledige verzameling X van veranderlijken in φ (arbitrair).

Claim

$\nu : X \rightarrow \{0, 1\}$ is een truth assignment dewelke φ waar maakt.

Bewijs: In elke clause is een literal waar gemaakt. □

Wanneer is een probleem even “hard” (moeilijk) als om het even welk probleem in NP?

Definition

Een taal B is **NP-hard** als voor elke taal $A \in \text{NP}$ we hebben dat $A \leq_P B$.

Wanneer is een probleem representatief voor NP?

Definition

Een taal B is **NP-compleet** als B NP-hard is en $B \in \text{NP}$.

Er zijn duizenden NP-complete problemen vanuit informatica, ingenieurswetenschappen, biologie, natuurkunde.

Er bestaan compendia met lijsten van NP-complete problemen.

Deterministisch polynomiale tijd (PTIME)

Satisfiability problemen

Niet-deterministisch polynomiale tijd (NP)

Reductie, compleetheid en hardheid

De stelling van Cook-Levin

NP-complete problemen

Maar is er nu een voorbeeld van een NP-compleet probleem?

Theorem

SAT is NP-compleet.

Herinner:

$$\text{SAT} = \{ \langle \varphi \rangle \mid \varphi \text{ is een satisfiable propositionele formule.} \}$$

We hebben al eerder aangetoond dat $\text{SAT} \in \text{NP}$.

We moeten aantonen dat SAT ook NP-hard is. M.a.w., elke taal A in NP is reduceerbaar tot SAT.

Opmerking: Dit toont opnieuw het belang van SAT aan. Een algoritme voor SAT kan gebruikt worden voor alle problemen in NP!

Bewijs. Neem om het even welke taal $A \in \text{NP}$

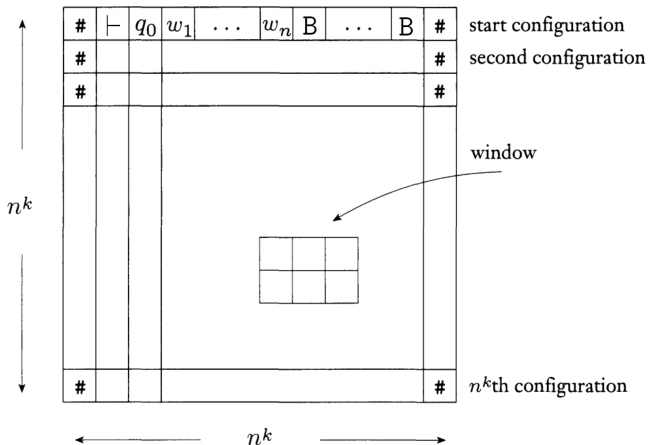
Stel, N is een NTM dewelke A beslist in $\mathcal{O}(n^\ell)$ voor een zeker ℓ . Meer precies, op een input van lengte n voert elke tak in de berekeningsboom van N op de input maximaal cn^ℓ stappen uit, voor een zekere constante c .

Neem nu een k zodat $cn^\ell + 3 \leq n^k$. We kunnen veronderstellen dat N $n^k - 3$ maximaal stappen doet (door eventueel N 's transitie relatie uit te breiden zodat er een tak is die inderdaad $n^k - 3$ stappen doet.)

We gaan een tak in de berekeningsboom van N op een input beschrijven door middel van een **tableau** van grootte $n^k \times n^k$.

Hierboven hebben we $n^k - 3$ gekozen omdat we twee cellen in het tableau voor speciale eindsymbolen gebruiken en één cel voor de toestand van N .

Een tableau:



Meer bepaald, een tableau is een $n^k \times n^k$ tabel, waarin

- ▶ rijen configuraties van N voorstellen op input w ;
- ▶ elke rij start en eindigt met een $\#$ symbool (dat niet in het alfabet zit);
- ▶ de eerste rij de initiële configuratie voorstelt;
- ▶ elke rij leidt tot de volgende rij (configuratie transitie); en
- ▶ een tableau is **aanvaardend** als er een rij bestaat die overeen komt met een aanvaardingsconfiguratie van N op w .

Duidelijk: N aanvaardt w als en slechts dan er een aanvaardend tableau van N op input w bestaat.

We moeten een reductie $f : \Sigma^* \rightarrow \Sigma^*$ maken van A naar SAT zodat $w \in A$ als en slechts dan $f(w) \in \text{SAT}$.

Voor een woord w maken we een $f(w) = \phi$ zodat:

- Er is een aanvaardend tableau N op input w als en slechts dan $f(w) = \phi$ satisfiable is.

De variabelen van ϕ worden gedefinieerd aan de hand van

$$C = Q \cup \Gamma \cup \{\#\},$$

waarin Q de toestanden en Γ het tape alfabet zijn van N .

Meer bepaald:

we nemen een variable $x_{i,j,s}$ voor elke index $i, j \in \{1, \dots, n^k\}$ en elk element $s \in C$

We noemen een element in een tableau een **cel**

- $\text{cel}[i, j]$ is de cel op positie (i, j) (i de rij, j de kolom)

De betekenis van $x_{i,j,s}$ is: $\nu(x_{i,j,s}) = 1$ als $\text{cel}[i, j] = s$; en anders $\nu(x_{i,j,s}) = 0$ voor een truth assignment ν .

De propositionele formule $f(w) = \phi$ gaat nu van de vorm zijn:

$$\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

We bespreken deze sub-formules nu in detail:

ϕ_{cell} : er bestaat een ν , $\nu \models \phi_{cell}$ als en slechts dan alle cellen er “ok” uit zien.

Meer bepaald, ϕ_{cell} is een formule die satisfiable gaat zijn wanneer elke cel in het tableau een unique symbool $s \in C$ is toegekend:

$$\phi_{cell} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s, t \in C \\ s \neq t}} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right]$$

Verder, ϕ_{start} zorgt ervoor dat de eerste rij de beginconfiguratie encodeert.

Meer bepaald, de formule ϕ_{start} is satisfiable als de eerste rij inderdaad overeenkomt met de beginconfiguratie van N op w :

$$\begin{aligned}\phi_{start} = & X_{1,1,\#} \wedge X_{1,2,\vdash} \wedge X_{1,3,q_0} \wedge \\ & X_{1,4,w_1} \wedge X_{1,5,w_2} \wedge \cdots \wedge X_{1,n+3,w_n} \wedge \\ & X_{1,n+4,\sqcup} \wedge \cdots \wedge X_{1,n^k-1,\sqcup} \wedge X_{1,n^k,\#},\end{aligned}$$

waarbij we veronderstellen dat $w = w_1 \cdots w_n$.

ϕ_{accept} drukt uit dat er ergens een aanvaardingstoestand in het tableau zit.

Meer bepaald, de formule ϕ_{accept} is waar wanneer er een rij is waarin de toestand een aanvaardingstoestand is:

$$\phi_{accept} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{accept}}.$$

Tenslotte, het moeilijkste is om een formule te maken die waar is als elke twee opeenvolgende rijen een configuratie overgang encodeert.

Dit gaan we doen door middel van de formule ϕ_{move} .

We moeten uitdrukken, d.m.v. een propositionele formule, dat twee opeenvolgende rijen in het tableau ook echt op elkaar volgen a.h.v. de transitierelatie van N .

We doen dit door het tableau te doorlopen d.m.v een “sliding window” van grootte 2×3 en na te kijken of de cellen in dit window **legaal** zijn.

Een window is legaal als het niet in het contradictie is met de transitierelatie van N .

Stel dat de transitierelatie δ van N bestaat uit:

- ▶ $\delta(q_1, a) = \{(q_1, b, \rightarrow)\}$ en
- ▶ $\delta(q_1, b) = \{(q_2, c, \leftarrow), (q_2, a, \rightarrow)\}$

Dan zijn dit "legal windows":

(a)

a	q_1	b
q_2	a	c

(b)

a	q_1	b
a	a	q_2

(c)

a	a	q_1
a	a	b

(d)

#		a
#		a

(e)

a	b	a
a	b	q_2

(f)

b	b	b
c	b	b

Inderdaad, ze beschrijven lokale transitities die niet in conflict zijn met δ .

Stel dat de transitierelatie δ van N bestaat uit:

- ▶ $\delta(q_1, a) = \{(q_1, b, \rightarrow)\}$ en
- ▶ $\delta(q_1, b) = \{(q_2, c, \leftarrow), (q_2, a, \rightarrow)\}$

De volgende windows zijn niet legaal:

(a)

a	b	a
a	a	a

(b)

a	q_1	b
q_1	a	a

(c)

b	q_1	b
q_2	b	q_2

Inderdaad, ze zijn in conflict met de transitierelatie.

Claim

*Als de bovenste rij in het tableau de beginconfiguratie representeert en **elk** window in het tableau legaal is, dan is elke rij in het tableau een configuratie en twee opeenvolgende rijen beschrijven een correcte overgang tussen de corresponderende configuraties.*

Bewijs Veronderstel, om een contradictie te bekomen, dat het tableau voldoet aan de condities in de claim, maar dat er toch twee rijen zijn, r_{i-1} and r_i die geen correctie transitie encoderen.

We nemen de eerste dergelijke rijen en dus r_{i-1} zal zeker een correcte configuratie zijn die bereikbaar is vanuit de beginconfiguratie.

Kijk nu in de rij r_{i-1} eerst naar cellen die niet op de rand liggen (en dus het #-symbool niet bevatten) en die niet naast een cell liggen die een toestand van N bevatten.

Neem zo'n cel $\text{cel}[i-1, j]$. Deze cel is deel van een legaal window gepositioneerd zodat $\text{cel}[i-1, j]$ "top center" is in dat window.

Aangezien er geen toestanden zijn in dergelijke windows, bestaan die cel uit twee kopiën van dezelfde informatie.

Als gevolg hiervan gaat $\text{cel}[i, j] = \text{cell}[i-1, j]$.

Er is nu een cell in r_{i-1} die een toestand bevat. Stel, dat dit cell $\text{cel}[i-1, k]$ is.

We weten nu dat r_i en r_{i-1} dezelfde inhoud hebben voor cellen op posities $1, \dots, k-2$ en $k+2, n^k$.

De fout zit dus in posities $k-1, k, k+1$ in rijen r_{i-1} en r_i !

Dit kan echter niet, omdat er een legaal window is over deze 6 cellen. □

Dit betekent dat we ϕ_{move} moeten definiëren zodat deze formule waar is als alle windows legal zijn:

$$\phi_{move} = \bigwedge_{1 \leq i < n^k, 1 \leq j < n^k} (\text{het } (i,j)\text{-de window is legaal})$$

De formule “het (i,j) -de window is legaal” is simpelweg de encoding van legale windows:

$$\bigvee_{\substack{a_1, \dots, a_6 \\ \text{is een legal window}}} (x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6})$$

De zes cellen in elk window kunnen op voorhand beschreven worden a.h.v. de transitierelatie δ en hangt niet af van w .

We hebben nu:

$$f(w) = \phi = \phi_{cel} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

Het is nu duidelijk dat ϕ satisfiable is als en slechts dan N het woord w aanvaard.

Resteert ons nog om aan te tonen dat de reductie $f : \Sigma^* \rightarrow \Sigma^*$ een **polynomiaal berekenbare functie** is.

Lemma

De reductie f is polynomiaal berekenbaar.

Bewijs. We tonen eerst aan dat de grootte van $f(w)$ polynomiaal is in n (anders zou $f(w)$ zowieso niet in polynomiale tijd berekend worden).

- ▶ Het aantal veranderlijken $x_{i,j,s}$ is $n^{2k} \times |C| = \mathcal{O}(n^{2k})$ omdat $|C|$ niet afhangt van de lengte van n .
- ▶ φ_{cell} : is van lengte $n^{2k} \times (|C| + |C|(|C| - 1)) = \mathcal{O}(n^{2k})$.
- ▶ φ_{start} : is van lengte $n^k = \mathcal{O}(n^k)$.
- ▶ φ_{accept} : is van lengte $n^{2k} = \mathcal{O}(n^{2k})$
- ▶ φ_{move} : is van lengte $n^{2k}|W|$ met W de verzameling van legal windows, en dit is onafhankelijk van n , dus ook $\mathcal{O}(n^{2k})$.
- ▶ Elke variable heeft $\mathcal{O}(\log(n))$ symbolen nodig:

Dus formule is $\mathcal{O}(\log(n)n^{2k})$ lang, nog steeds polynomiaal in n . ($\mathcal{O}(\log(n))$ is kleiner dan $\mathcal{O}(n)$)

Het is duidelijk dat we $f(w)$ kunnen berekenen door een algoritme dat in polynomiale tijd loopt. $\square$

Deterministisch polynomiale tijd (PTIME)

Satisfiability problemen

Niet-deterministisch polynomiale tijd (NP)

Reductie, compleetheid en hardheid

De stelling van Cook-Levin

NP-complete problemen

Stel, we willen aantonen dat CNF-SAT en 3SAT ook NP-compleet zijn.

- We kunnen opnieuw, van scratch, aantonen dat elke polynomiale tijd NTM reduceerbaar is naar deze problemen.

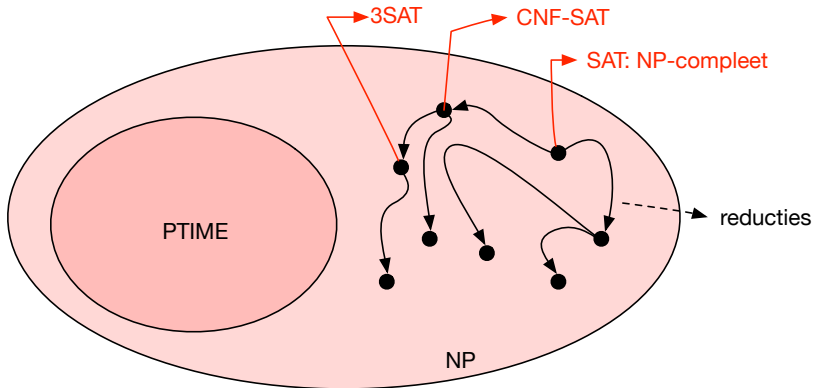
Echter, kunnen we het werk dat we gedaan hebben voor SAT (de Stelling van Cook-Levin) niet **hergebruiken**?

Ja, door gebruik te maken van **reducties**.

Propositie

1. *(Transitiviteit) Als $A \leq_P B$ en $B \leq_P C$ dan $A \leq_P C$.*
2. *Als A NP-compleet is en $A \leq_P B$, voor een $B \in NP$, dan is ook B NP-compleet.*

Je kan elk gekend NP-compleet probleem gebruiken om aan te tonen dat jouw probleem in NP ook NP-compleet is.



Ze zijn transitief:

Propositie

Als $A \leq_P B$ en $B \leq_P C$ dan ook $A \leq_P C$.

Bewijs Stel, f_1 is een reductie van A naar B , f_2 is een reductie van B naar C . Beschouw dan de samenstelling $g : f_2 \circ f_1$, dit is een functie van A naar C .

Meer nog, $w \in A$ als en slechts dan $f_1(w) \in B$ als en slechts dan $g(w) = f_2(f_1(w)) \in C$. $\square$

We kunnen dus reducties samenstellen.

Propositie

If A is NP-complete and $A \leq_P B$, for $B \in NP$, then B is NP-complete.

Bewijs We moeten aantonen dat B NP-hard is.

Stel, f_1 is een reductie van een NTM N naar A , f_2 is een reductie van A naar B .

Dan is ook $f_2 \circ f_1$ een reductie van N naar B . □

Om dus aan te tonen dat CNF-SAT en 3SAT ook NP-compleet zijn:

- ▶ Toon aan dat ze in NP zitten (dit is vanzelfsprekend aangezien dit deelproblemen zijn van SAT).
- ▶ Maak een reductie van SAT naar CNF-SAT (zodat CNF-SAT ook NP-compleet is).
- ▶ Maak een reductie van CNF-SAT naar 3SAT (zodat 3SAT ook NP-compleet is).

Gegeven een propositionele formule φ over veranderlijken X , transformeren we $\langle \varphi \rangle$ door middel van een polynomiale reductie $f : \Sigma^* \rightarrow \Sigma^*$ naar $\langle \text{cnf}(\varphi) \rangle$, een propositionele formule in conjunctieve normaalvorm, zodat

$$\langle \varphi \rangle \in \text{SAT} \text{ als en slechts dan } \langle \text{cnf}(\varphi) \rangle \in \text{CNF-SAT}.$$

We hebben reeds gezien hoe dit kan zodat $\text{cnf}(\varphi)$ van polynomiale lengte is in de lengte van φ door nieuwe veranderlijken te gebruiken.

Propositie

CNF-SAT is NP-compleet.

Gegeven een CNF formule φ over veranderlijken X , transformeren we $\langle \varphi \rangle$ door middel van een polynomiale reductie $f : \Sigma^* \rightarrow \Sigma^*$ naar $\langle \psi \rangle$, een CNF formule ψ met drie veranderlijken per clause en zodat

$$\langle \varphi \rangle \in \text{CNF-SAT} \text{ als en slechts dan } \langle \psi \rangle \in 3\text{SAT}.$$

We doen dit door opnieuw nieuwe veranderlijken te gebruiken.

We moeten er voor zorgen dat elke clause C in φ uit drie veranderlijken bestaat.

Voorbeeld

Stel, $C = (a \vee b \vee c \vee d \vee e)$ dan transformeren we deze clause in

$\tilde{C} = (a \vee z_1 \vee z_1) \wedge (\neg z_1 \vee b \vee z_2) \wedge (\neg z_2 \vee c \vee z_3) \wedge (\neg z_3 \vee d \vee z_4) \wedge (\neg z_4 \vee \neg z_4 \vee e).$
met nieuwe veranderlijken z_1, z_2, z_3 en z_4 .

Dit is duidelijke een polynomiale tijd transformatie.

Is dit echter correct?

Stel, algemeen $\varphi = C_1 \wedge \dots \wedge C_m$ met $C_i = \ell_1^i \vee \ell_2^i \vee \dots \vee \ell_{p_i}^i$. Dan,

$$C'_i = (z_1^i \vee z_1^i \vee \ell_1^i) \wedge \bigwedge_{j=2}^{p_i-1} (\neg z_{j-1}^i \vee \ell_j^i \vee z_j^i) \wedge (\neg z_{p_i-1}^i \vee \neg z_{p_i-1}^i \vee \ell_{p_i}^i)$$

en

$$\psi = C'_1 \wedge C'_2 \wedge \dots \wedge C'_m.$$

Neem nu een truth assignment $\nu : X \rightarrow \{0, 1\}$ zodat $\nu \models \varphi$.

Dit wil zeggen dat $\nu \models C_i$ voor elke $i = 1, \dots, m$.

Er is dus minstens één literal, stel ℓ_k^i in C_i zodat $\nu(\ell_k^i) = 1$.

Er is een corresponderende clause in C'_i van de vorm $(\neg z_{k-1}^i \vee \ell_k^i \vee z_k^i)^1$. Zet $\nu'(\neg z_{k-1}^i) = 0$ en $\nu'(z_k^i) = 0$ en $\nu'(\ell_k^i) = 1$.

Aangezien z_{k-1}^i en $\neg z_k^i$ respectievelijk in de clause ervoor en de clause erna voorkomen, kunnen we daarin opnieuw $\nu'(\neg z_{k-2}^i) = 0$ en $\nu'(z_{k+1}^i) = 0$ kiezen. Op deze manier $\nu' \models C'_i$.

Dit resulteert in een truth assignment $\nu' \models \psi$.

¹De gevallen $k = 1$, $k = p_i$ zijn gelijkaardig.

Voorbeeld

Stel, $C = (a \vee b \vee c \vee d \vee e)$ met $\nu(a) = \nu(b) = \nu(d) = \nu(e) = 0$ en $\nu(c) = 1$, voor $C' = (a \vee z_1 \vee z_1) \wedge (\neg z_1 \vee b \vee z_2) \wedge (\neg z_2 \vee c \vee z_3) \wedge (\neg z_3 \vee d \vee z_4) \wedge (\neg z_4 \vee \neg z_4 \vee e)$ definiëren we:

$$\nu'(c) = 1, \nu'(z_2) = 1, \nu'(z_3) = 0$$

$$\nu'(d) = 0, \nu'(z_4) = 0$$

$$\nu'(e) = 0$$

$$\nu'(b) = 0, \nu'(z_1) = 1$$

$$\nu'(a) = 0$$

Omgekeerd, neem $\nu' \models \psi$.

Elke z_j^i kan slechts één enkele clause in C_i' waar maken, maar we hebben één clause meer dan het aantal z_j^i 's.

Dus, er moet een clause zijn in C_i' die waar wordt gemaakt door een oorspronkelijke literal.

Definiëer nu $\nu(x_i) = \nu'(x_i)$ voor veranderlijken in X .

Dus, door $f : \Sigma^* \rightarrow \Sigma^*$ te nemen als

$$\langle \varphi \rangle \rightarrow f(\langle \varphi \rangle) = \langle \psi \rangle$$

hebben we reductie van CNF-SAT naar 3SAT.

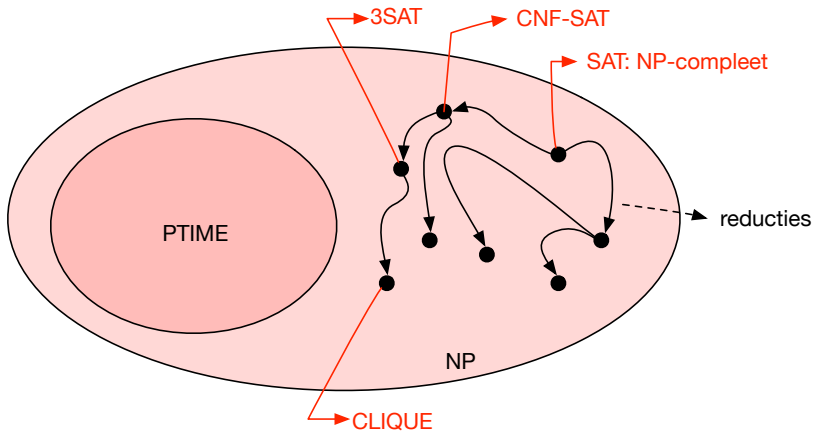
Propositie

3SAT is NP-compleet.

Corollary

CLIQUE is NP-compleet.

CLIQUE is NP-complete



Definition

VERTEX-COVER = $\{ \langle G, k \rangle \mid G \text{ is een ongerichte graaf met een vertex cover bestaande uit } k \text{ knopen} \}$.

Een **vertex cover** van G is een deelverzameling van de knopen van G zodat elke boog in G een eindpunt heeft in die deelverzameling.

Theorem

*Neem een ongerichte graaf $G = (V, E)$. Een verzameling van knopen C is een k -clique in G als en slechts dan $V \setminus C$ een $|V| - k$ -vertex cover is in de **complement graaf** $\bar{G}$ van G .*

De **complement graaf** $\bar{G}$ van $G = (V, E)$ is gedefinieerd als $(V, E') = (V, V^2 \setminus E)$.

Theorem

Neem een ongerichte graaf $G = (V, E)$. Een verzameling van knopen C is een k -clique in G als en slechts dan $V \setminus C$ een $|V| - k$ -vertex cover is in de complement graaf $\bar{G}$ van G .

Bewijs. Stel dat C een k -clique is in $G = (V, E)$.

Beschouw $C' = V \setminus C$.

Neem een boog $(v, w) \in E'$. Dan is $(v, w) \notin E$.

En dus moet ofwel v of w niet in C zitten (C is een clique).

En ofwel v of w zit in C' (C' is het complement van C)

C' bevat dus een knoop voor elke boog in E' en is dus een vertex cover van grootte $|V| - k$.

Omgekeerd, beschouw een vertex cover C' van $\bar{G}$ van grootte $|V| - k$.

Beschouw $C = V \setminus C'$ (grootte k)

Stel, $v, w \in C$ maar $(v, w) \notin E$.

Dan, $(v, w) \in E'$ en zowel v als w zitten niet in C' (omdat $v, w \in C$)

Maar dan is C' geen vertex cover. Een contradictie.

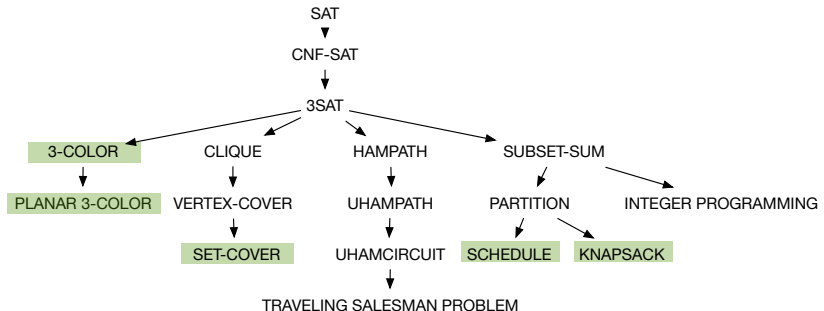
C bevat dus een boog tussen elke twee knopen en is dus een clique van grootte k .

We kunnen dus een reductie maken van CLIQUE naar VERTEX-COVER door $\langle G, k \rangle$ af te beelden op $\langle \bar{G}, |V| - k \rangle$.

Propositie

VERTEX-COVER is NP-compleet.

(Toon zelf aan dat VERTEX-COVER in NP zit.)



- ▶ Een pijltje betekend reduceerbaar naar
- ▶ De groene problemen worden in de oefeningen besproken.

Het vinden van een reductie is een kunst en is zoals het vinden van een algoritme voor een probleem door middel van een algoritme voor een ander probleem.

Herinner

$HAMPATH = \{ \langle G, s, t \rangle \mid G \text{ is a gerichte graaf met} \\ \text{een Hamiltoniaans pad van } s \text{ naar } t \}.$

Theorem

HAMPATH is NP-compleet

Bewijs: We weten al dat HAMPATH in NP zit.

Om NP-hardheid aan te tonen gebruiken we een reductie van 3SAT naar HAMPATH.

Voor een 3CNF formule φ construeren we een gerichte graaf met knopen s en t zodat φ is satisfiable als en slechts dan G een Hamiltoniaans pad heeft van s naar t .

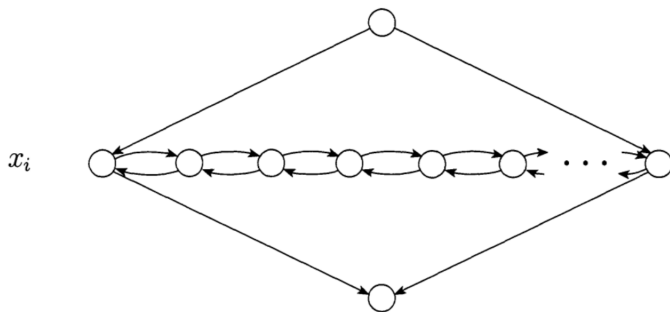
Dus, gegeven

$$\varphi = \underbrace{(u_1 \vee v_1 \vee w_1)}_{c_1} \wedge \underbrace{(u_2 \vee v_2 \vee w_2)}_{c_2} \wedge \cdots \wedge \underbrace{(u_k \vee v_k \vee w_k)}_{c_k}$$

met daarin

- ▶ variabelen u_i, v_i, w_i van de vorm x_i of $\overline{x_i}$;
- ▶ $x_1, \dots, x_\ell$ zijn alle veranderlijken in φ ; en
- ▶ $c_1, \dots, c_k$ noteren de k clauses van φ .

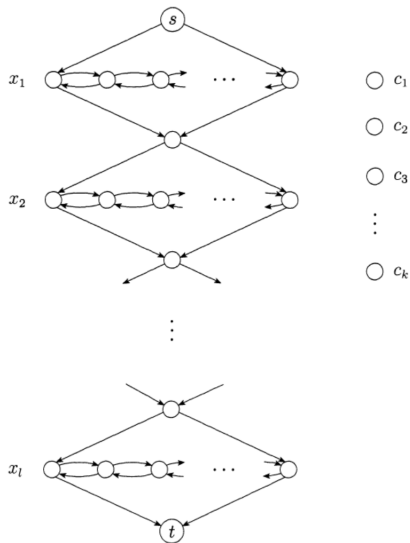
Veranderlijke x_i wordt gerepresenteerd door een diamant structuur:



en elke clause c_j door één enkele knoop:



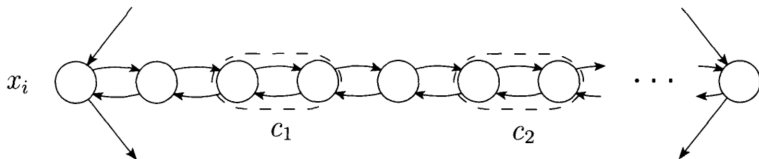
De initiële structuur van de graaf G en knopen s en t is als volgt:



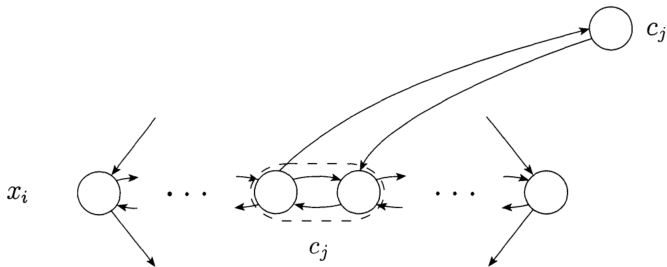
We gaan nu de veranderlijken (diamant structuren) aan de clauses (enkele knopen) hangen met bogen.

Hier gaan we gebruik maken van de horizontale rij van diamanten voor elke veranderlijke. Deze bestaan uit:

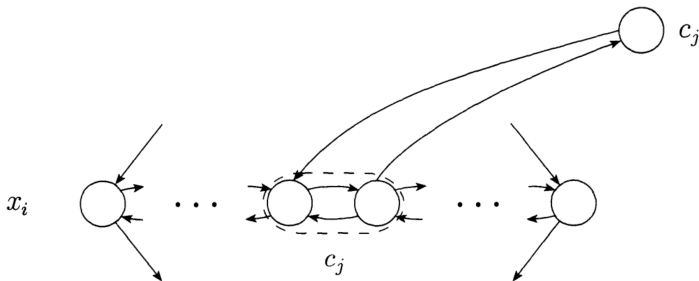
- ▶ $3k + 3$ knopen (k is het aantal clauses);
- ▶ Twee op het eind (links, rechts), en $3k + 1$ knopen binnenin; en
- ▶ We gaan een paar knopen gebruiken voor elke clause (dus $2k$ knopen in totaal), en elk van deze paren zijn gescheiden met een knoop ($k + 1$).



Als x_i in de clause c_j zit doen we hetvolgende:



Als $\bar{x}_i$ in de clause c_j zit doen we het volgende:



De graaf G kan duidelijk bekomen worden in polynomiale tijd gegeven $\langle \varphi \rangle$.

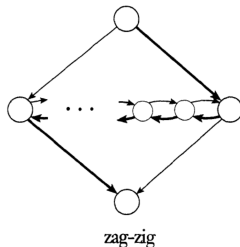
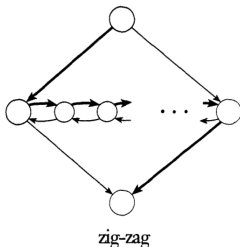
We moeten nog aantonen dat dit een reductie is!

φ is satisfiable als en slechts dan G een Hamiltoniaans pad heeft van s naar t

Veronderstel dat φ satisfiable is. Er is dus een truth assignment $\nu : X \rightarrow \{0, 1\}$ die φ waar maakt.

Hamiltonian pad: start in s en:

- ▶ zig-zag als $\nu(x_i) = 1$
- ▶ zag-zig als $\nu(x_i) = 0$



Op deze manier bezoeken we elke knoop éénmaal. (knoten voor clauses volgen later.)

We breiden het bekomen pad nu uit zodat ook elke clause knoop wordt bezocht.

- ▶ Kies in elke clause één literal die de clause waar maakt.
- ▶ Als x_i gekozen wordt voor c_j maak een detour op de zig-zag voor x_i via de knoop voor c_j en dit dmv j de paartje op de horizontale diamant
- ▶ Als $\bar{x}_i$ gekozen wordt voor c_j maak een detour op de zag-zig voor $\bar{x}_i$ via de knoop voor c_j en dit dmv j paartje op de horizontale diamant

Door de constructie loopt het pad (zig-zag of zag-zig) in de juiste richting om zo een correct pad te bekomen. Het is duidelijk dat dit een Hamiltoniaans pad is.

Omgekeerd, veronderstel we een Hamiltoniaans pad hebben van s naar t

Dergelijk pad:

- ▶ moet beginnen in s (geen inkomende bogen) en eindigen in t (geen uitgaande bogen)
- ▶ moet elke diamant bezoeken

Stel, het pad begint aan een zig-zag (zag-zig is hetzelfde argument). Dan zal het de volledige horizontale diamant moeten bezoeken in deze gekozen richting

- ▶ Dit is vanzelfsprekend als er geen clause knopen waren.

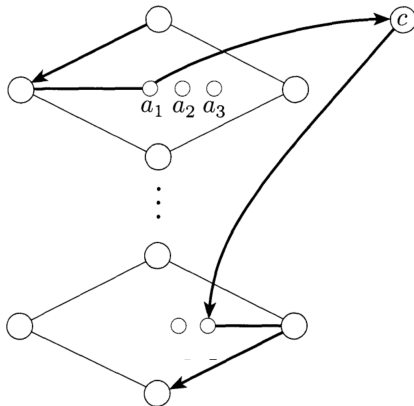
Wanneer we ook de clause knopen bekijken:

Wanneer het pad zoals in het vorige deel was van het bewijs kunnen we hier eenvoudig een satisfying truth assignment voor φ uithalen

- ▶ zig-zag voor x_i dan $\nu(x_i) = \text{true}$
- ▶ zag-zig voor x_i dan $\nu(x_i) = \text{false}$

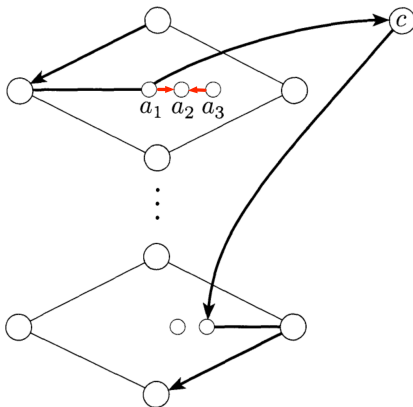
We tonen nu aan dat elk Hamiltoniaans pad zo'n goed pad is.

Veronderstel dat op de horizontale diamant er vanuit een knoop a_1 naar een clause knoop c gegaan wordt, en vanuit c naar een knoop gaat die verschillend is van de knoop a_2 rechts van a_1 op de horizontale diamant.



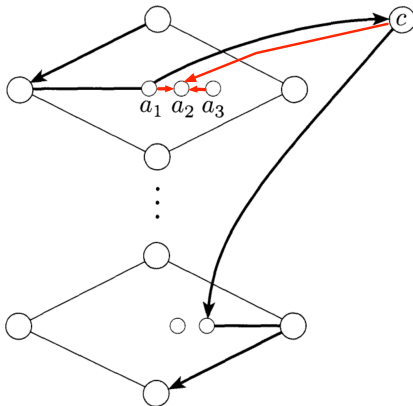
Dit kan echter niet voor een Hamiltoniaans pad.

Ofwel is a_1 de tweede knoop voor de clause c in het diamant.



En dus is er geen manier om tot a_2 te geraken zonder vast te zitten.

Ofwel is a_1 de eerste knoop voor de clause c in het diamant.



En dus is er opnieuw geen manier om tot a_2 te geraken zonder vast te zitten.

Dus, een Hamiltoniaans pad moet van a_1 naar a_2 gaan via c .



Definition

UHAMPATH = $\{\langle G, s, t \rangle \mid G \text{ is a ongerichte graaf met}$
een Hamiltoniaans pad van s naar $t\}$.

Theorem

UHAMPATH is NP-compleet.

Bewijs: We maken een reductie van HAMPATH naar UHAMPATH.
Meer bepaald, we construeren een ongerichte graaf G' gegeven G als volgt:

- ▶ s wordt s^{out} , t wordt t^{in} ; en
- ▶ elke andere knoop u worden drie knopen: u^{in} , u^{mid} en u^{out} .

Voor de bogen:

- Voor elke knoop u in G : (u^{mid}, u^{in}) en (u^{mid}, u^{out}) worden bogen in G' .
- Voor elke boog (u, v) in G : (u^{out}, v^{in}) wordt een boog in G' .

en dat is het.

Stel nu dat $s, u_1, u_2, \dots, u_k, t$ een Hamiltoniaans pad is in G van s naar t . Dan, beschouw het pad

$$s^{out}, u_1^{in}, u_1^{mid}, u_1^{out}, u_2^{in}, u_2^{mid}, u_2^{out}, \dots, u_k^{out}, t^{in}.$$

Dit is duidelijk ook een Hamiltoniaans pad in G' .

Omgekeerd, neem een Hamiltoniaans pad P' in G' . Dit pad start in s^{out} en moet eindigen in t^{in} .

- ▶ De enige knopen verbonden met s^{out} zijn van de vorm u_i^{in} . Dus $P' = s, u_{i_1}^{in}, \dots$ voor een zekere i_1 .
- ▶ Dit pad moet ook $u_{i_1}^{mid}$ bevatten. Aangezien deze knoop slechts verbonden is met bogen met $u_{i_1}^{in}$ en $u_{i_1}^{out}$ moet $P' = s, u_{i_1}^{in}, u_{i_1}^{mid}, \dots$
- ▶ De enige knoop nog verbonden (en nog niet bezocht) met $u_{i_1}^{mid}$ is $u_{i_1}^{out}$. Dus, $P' = s, u_{i_1}^{in}, u_{i_1}^{mid}, u_{i_1}^{out}, \dots$
- ▶ Herhaal dit nu, startend vanaf $u_{i_1}^{out}$.

Dus,

$$P' = s^{out}, u_{i_1}^{in}, u_{i_1}^{mid}, u_{i_1}^{out}, u_{i_2}^{in}, u_{i_2}^{mid}, u_{i_2}^{out}, \dots, u_{i_k}^{out}, t^{in}.$$

waaruit we opnieuw een Hamiltoniaans pad $s, u_{i_1}, u_{i_2}, \dots, u_{i_k}, t$ in G kunnen halen.



- Een **cycle** in een graaf is een pad zonder herhaling van de knopen, die begint en eindigt in dezelfde knoop.

Definition

UHAMCIRCUIT = $\{\langle G \rangle \mid G \text{ is een ongerichte graaf}$
waarin een cycle zit die elke knoop in G eenmaal bezoekt $\}$.

Theorem

UHAMCIRCUIT is NP-compleet

Bewijs: Het is duidelijk dat dit in NP zit. Genereer een cycle en test of deze Hamiltoniaans is.

Voor NP-hardheid: We maken een reductie van UHAMPATH naar UHAMCIRCUIT.

Eenvoudig: Gegeven $\langle G, s, t \rangle$ maak een nieuwe graaf G' met een extra knoop x en ongerichte bogen (x, s) en (x, t) . $\square$

- ▶ Een **volledige** graaf $G = (V, E)$ is een graaf met bogen tussen **alle knopen** (bijvoorbeeld, knopen=steden, bogen=wegen tussen steden).
- ▶ Een **gewogen** graaf $G = (V, E, c)$ is een graaf met een **kost functie** $c : E \rightarrow \mathbb{N}$ (bijvoorbeeld, kost=afstand tussen knopen) op de bogen.
- ▶ De **kost van een cycle** is de som van de kost van elke boog in de cycle.

Definition

TSP = $\{ \langle G, c, k \rangle \mid G \text{ is een volledige gewogen graaf} \\ \text{waarin een cycle zit die elke knoop eenmaal bezoekt met kost } \leq k. \}$.

TSP=Travelling Salesman Problem, een verkoper moet alle steden bezoeken (éénmaal) door weinig kilometers te doen ($\leq k$).

Theorem

TSP is NP-compleet.

Bewijs: Dit zit duidelijk in NP: Genereer een cycle, test of deze Hamiltoniaans is en een kost heeft kleiner dan k .

Voor NP-hardheid: We maken een reductie van UHAMCIRCUIT naar TSP.

Opnieuw eenvoudig: Gegeven $\langle G \rangle$ maken we $\langle G', c, k \rangle$ met

- ▶ $G' = (V, E')$ de volledige graaf over de knopen in $G = (V, E)$.
- ▶ $c : E' \rightarrow \mathbb{N}$ zodat

$$c(u, v) = \begin{cases} 1 & \text{als } (u, v) \text{ geen boog in } E \text{ is} \\ 0 & \text{anders.} \end{cases}$$

- ▶ $k = 0$.

Het is duidelijk dat wanneer er Hamiltoniaans circuit C in G is, dat dit dan een oplossing vormt voor TSP in G' met een kost gelijk aan 0.

Omgekeerd, stel we hebben een TSP oplossing C in G' van kost 0, dan moet deze cycle bestaan uit bogen in G , en dus dit is ook een Hamiltoniaanse cycle in G . □

Een verzameling (set) $S = \{x_1, \dots, x_k\}$ bestaat uit verschillende elementen. In een **multiset** $S = \{\{x_1, \dots, x_k\}\}$ mogen elementen meerdere keren voorkomen.

Definition

SUBSETSUM = $\{\langle S, t \rangle \mid S = \{\{x_1, \dots, x_k\}\}$ en er is een deel-multiset $T = \{\{y_1, \dots, y_\ell\}\}$ zodat $\sum_{i=1}^{\ell} y_i = t\}$.

Theorem

SUBSETSUM is NP-complete.

Bewijs: Dat het in NP zit: “gok” een deel-multiset T en test of de sum van de elementen gelijk is aan t .

Om NP-hardheid aan te tonen maken we een reductie van 3SAT naar SUBSET-SUM

Dus, neem een 3CNF formule φ met:

- ▶ $x_1, \dots, x_\ell$ de veranderlijken in φ
- ▶ $c_1, \dots, c_k$ de clauses van φ

De multiset S bestaat uit getallen:

- ▶ $y_1, z_1, \dots, y_\ell, z_\ell$ om de veranderlijken te representeren; en
- ▶ $g_1, h_1, \dots, g_k, h_k$ om de clauses te representeren.

De volgende tabel beschrijft de getallen in S in **decimale notatie** en de doelwaarde t

Voor $\varphi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_2 \vee x_3 \vee \dots) \wedge \dots \wedge (\overline{x_3} \vee \dots \vee \dots)$

	1	2	3	4	...	l	c_1	c_2	...	c_k
y_1	1	0	0	0	...	0	1	0	...	0
z_1	1	0	0	0	...	0	0	0	...	0
y_2		1	0	0	...	0	0	1	...	0
z_2		1	0	0	...	0	1	0	...	0
y_3			1	0	...	0	1	1	...	0
z_3			1	0	...	0	0	0	...	1
$\vdots$					$\ddots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
y_l						1	0	0	...	0
z_l						1	0	0	...	0
g_1							1	0	...	0
h_1							1	0	...	0
g_2								1	...	0
h_2								1	...	0
$\vdots$									$\ddots$	$\vdots$
g_k										1
h_k										1
t	1	1	1	1	...	1	3	3	...	3

Dus in deze tabel bestaan de getallen y_i en z_i uit twee delen:

- ▶ Links bestaan ze uit een 1 gevolgd door $\ell - i$ nullen;
- ▶ Rechts heeft y_i een 1 in positie j als x_i in de clause c_j zit, en 0 anders;
- ▶ Rechts heeft z_i een 1 in positie j als $\overline{x_i}$ in de clause c_j zit, en 0 anders.

Daarentegen, de getallen g_j, h_j zijn gelijk aan elkaar

- ▶ een 1 gevolgd door $k - j$ nullen.

Tenslotte, t bestaat uit ℓ enen gevold door k drieën.

Deze reductie is duidelijk polynomiaal berekenbaar. Is het zo dat φ satisfiable is als en slechts dan S een deel-multiset heeft met elementen die sommeren tot t ? Veronderstel dat φ satisfiable is en zij $\nu : X \rightarrow \{0, 1\}$ een truth assignment die φ waar maakt.

- ▶ Als $\nu(x_i) = 1$, dan stop het getal y_i in T , anders stop het getal z_i erin.
- ▶ Dit zorgt er al voor dat na sommeren een 1 krijgen in de eerste ℓ posities.
- ▶ Elke clause heeft ten minste één en maximaal drie literals die waar worden. Na sommatie liggen de laatste k getallen tussen 1 en 3.
- ▶ Om dus tot 3 te komen in elk van de k posities, moeten we dus genoeg getallen selecteren uit $\{g_i, h_i\}$ (geen, één, of twee).

Dit leidt tot een oplossing van SUBSETSUM.

Omgekeerd, stel dat T een oplossing van SUBSETSUM.

- ▶ In T kan slechts y_i ofwel z_i zitten (anders > 1), maar minstens één van de twee moet in T (anders < 1 omdat we met kleine getallen werken en dus geen overdracht hebben tijdens sommeren).
- ▶ We definiëren $\nu(x_i) = 1$ als $y_i \in T$, anders zetten we $\nu(x_i) = 0$.
- ▶ In de laatste k kolommen hebben we steeds de waarde 3.
- ▶ Zo een 3 moet bekomen zijn door minstens een y_i of z_i in T (g_i en h_1 sommeren maar tot 2)
- ▶ Dit betekent dat er voor elke clause een literal bestaat die die clause waar maakt.

En dus, $\nu \models \varphi$ en φ is satisfiable.



Beschouw een multiset S van positieve getallen (natuurlijke getallen)

Definition

PARTITION = $\{\langle S \rangle \mid S = \{\{x_1, \dots, x_k\}\}$ and we kunnen S opsplitsen in S_1 en S_2 zodat de sommen van hun elementen gelijk zijn $\}$.

Theorem

PARTITION is NP-complete.

Bewijs. Het zit duidelijk in NP: Genereer twee verzamelingen S_1 en S_2 , check of deze een partitie vormen van S en $\sum_{x \in S_1} x = \sum_{x \in S_2} x$.

Voor NP-hardheid, tonen we aan dat $\text{SUBSET-SUM} \leq_P \text{PARTITION}$.

Gegeven $\langle S, t \rangle$ construeren we $\langle S' \rangle$ zodat

$$\blacktriangleright S' = S \cup \{y_1 = 2 \sum_{x \in S} x - t\} \cup \{y_2 = \sum_{x \in S} x + t\}.$$

Een oplossing T zodat $\sum_{x \in T} x = t$ leidt tot $\sum_{x \in S \setminus T} x = \sum_{x \in S} x - t$.

Voeg nu y_1 toe aan T om zo S_1 te krijgen met

$$\sum_{x \in S_1} x = t + 2 \sum_{x \in S} x - t = 2 \sum_{x \in S} x.$$

Voeg y_2 toe aan $S \setminus T$ om zo S_2 te krijgen met

$$\sum_{x \in S_2} x = \sum_{x \in S} x - t + \sum_{x \in S} x + t = 2 \sum_{x \in S} x.$$

Omgekeerd, stel dat we S_1 en S_2 hebben zodat

$$\sum_{x \in S_1} x + \sum_{x \in S_2} x = \sum_{x \in S} x + 2 \sum_{x \in S} x - t + \sum_{x \in S} x + t = 4 \sum_{x \in S} x.$$

Dan, y_1 moet ofwel in S_1 of S_2 zitten. Stel dat $y_1 \in S_1$. Merk op, $y_2 \notin S_1$. We hebben

$$\sum_{x \in S_1 \setminus \{y_1\}} x = 2 \sum_{x \in S} x - 2 \sum_{x \in S} x + t = t,$$

zoals gewenst. □

Een **integer program** wordt gegeven door een $m \times n$ matrix A bestaande uit gehele getallen en een $m \times 1$ vector b , ook bestaande uit gehele getallen.

Een **oplossing** van een integer program (A, b) is een **binaire** $n \times 1$ vector x zodat

$$A \cdot x = b,$$

ofwel, $x_j \in \{0, 1\}$ en $\sum_{j=1}^n a_{ij}x_j = b_i$ voor alle $i = 1, \dots, m$ en $j = 1, \dots, n$.

Definition

IP = $\{ \langle A, b \rangle \mid \text{het gehele programma } A \cdot x = b \text{ heeft een oplossing} \}$.

Theorem

IP is NP-complete.

Bewijs. In NP, eenvoudig. Voor NP-hardheid tonen we aan dat SUBSET-SUM $\leq_P$ IP.

Gegeven $\langle S, t \rangle$, beschouwen we $\sum_{a_i \in S} a_i x_i = t$ met $x_i \in \{0, 1\}$. Een oplossing hiervan geeft duidelijk een oplossing voor SUBSET-SUM, en omgekeerd. $\square$

Gegeven een probleem A kunnen we ook naar het bijhorende **complement** beslissingsprobleem $w \notin A$ kijken.

Voorbeeld

$\overline{\text{SAT}}$ (unsatisfiable formulae), $\overline{\text{CLIQUE}}$ (geen cliques),...

Het is niet zo dat als $A \in \text{NP}$ zit, dat dan ook $A \in \text{coNP}$ zit!

coPTIME en coNP zijn de tijdscomplexiteitsklassen voor problemen waarvoor hun complement in PTIME en NP zitten.

Het is duidelijk dat $\text{coPTIME} = \text{PTIME}$.

Het is niet geweten of NP en coNP verschillen!

Wel zit PTIME zowel in NP als coNP . En er bestaan problemen die compleet zijn voor de klasse $\text{NP} \cap \text{coNP}$.

We beschouwen coNP niet verder in deze cursus, maar dit komt aan bod in het project.

We kunnen nu polynomiale reducties gebruiken tussen problemen in NP.
Wat is het effect van dergelijke reducties op problemen in PTIME?

Propositie

Als $A \leq_P B$ en $B \in PTIME$ dan is ook $A \in PTIME$.

Bewijs. Oefening. □

Als we dus een reductie kunnen van een NP-compleet probleem naar een polynomiaal probleem dan is dus $NP = PTIME$.

Dit is dus een goede manier om problemen in NP en PTIME te onderscheiden.

Stel, we definiëren dit als volgt:

Een probleem A is PTIME-compleet als:

- ▶ A zit in PTIME; en
- ▶ Voor elk ander probleem $B \in \text{PTIME}$ hebben we dat $B \leq_P A$.

Goed idee?

Kunnen we hiermee problemen in PTIME van elkaar onderscheiden?

In deze definitie gaat om het even welke taal $A \neq \emptyset, \Sigma^*$ in PTIME, ook PTIME-compleet zijn.

Inderdaad, neem een taal $B \in \text{PTIME}$ en definieer de reductie $f : \Sigma^* \rightarrow \Sigma^*$ zodat

$$f(w) = \begin{cases} w_1 \in A & w \in B \\ w_2 \notin A & w \notin B, \end{cases}$$

waarin w_1 en w_2 vastgekozen woorden zijn in A , respectievelijk niet in A .

Dit is alles berekenbaar in polynomiale tijd.

Elke probleem in PTIME is dus PTIME-compleet met deze definitie.

Reden is dat **polynomiale** reducties te krachtig zijn. We komen hier later nog op terug.

- ▶ PTIME klasse en begrijpen van algoritmen voor PTIME problemen (Sipser 7.2)
- ▶ NP klasse (Sipser 7.3)
- ▶ Reducties, hardheid en compleetheid (Sipser 7.4)
- ▶ De Stelling van Cook-Levin, inclusief het bewijs (Sipser 7.4)
- ▶ NP-hard problemen, en begrijpen van de reducties voor de geziene problemen (Sipser 7.5).